

NMV - MU5IN453 :

Git, un gestionnaire de versions décentralisé.

Version 20.02

Julien Sopena¹

¹julien.sopena@lip6.fr
Équipe Delys - INRIA Paris
LIP6 - Sorbonne Université / CNRS

Master SAR 2^{ème} année - 2023/2024

Git c'est quoi ?

Architecture interne de git

- Création d'un dépôt
- Les objets manipulés par git
- Les "Blobs" (fichier)
- Les "Trees"
- Les "Commits"
- Les "Tags"
- Structure générale

Gestion des versions dans le dépôt local.

- Les commits
- Les branches
- Les merges

Les rebases

Les remords

Utilisation de l'historique

Synchronisation avec les dépôts distants.

Principe des DVCS :

Gestionnaire de versions décentralisé.

Les dépôts distants.

Modèles de travail coopératif

Les outils graphiques

git-gui et gitk

Exemple de gitg

Conclusion

Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

Principe de base des gestionnaires de version

Un gestionnaire de versions (**VCS**) doit permettre de :

- ▶ conserver toutes les versions de tous les fichiers ;
- ▶ conserver toutes les arborescences de fichiers ;
- ▶ permettre d'identifier une arborescence de versions de fichiers ;
- ▶ fournir des outils pour gérer le tout.

Un **DVCS** ("*Distributed Version Control*") offre les mêmes services sur une architecture décentralisée.

Tous deux reposent sur deux mécanismes :

1. un mécanisme permettant de calculer la différence entre deux versions : **diff/patch** ;
2. un gestionnaire d'**historique** des diff.

diff & patch

diff : Comparaison de fichiers ligne par ligne

- ▶ indique les lignes ajoutées ou supprimées ;
- ▶ peut ignorer les casses, les tabulations, les espaces ;
- ▶ option **-u** pour créer des patches unifiés, avec plus d'informations.

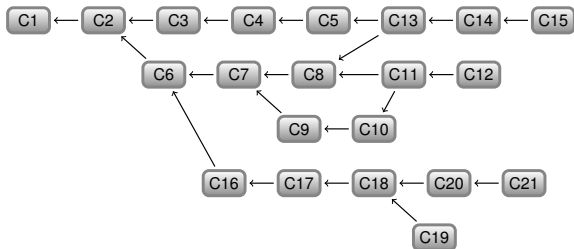
patch : Utilise la différence entre deux fichiers pour passer d'une version à l'autre.

Exemple

```
$ diff toto.c toto-orig.c > correction.patch  
$ bzip2 correction.patch  
$ bzipcat correction.patch.bz2 | patch -p 0 toto.c
```

Definition

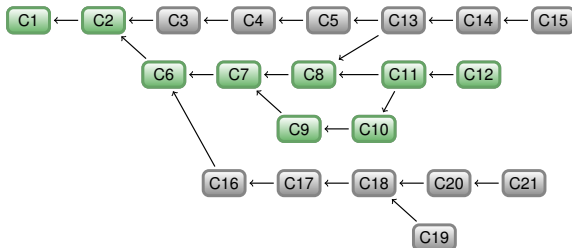
On appelle **historique** un graphe orienté acyclique composé d'un ensemble de versions pouvant être recalculées à partir des versions adjacentes en appliquant les patches modélisés par les arcs sortants.



Historique : Les branches

Definition

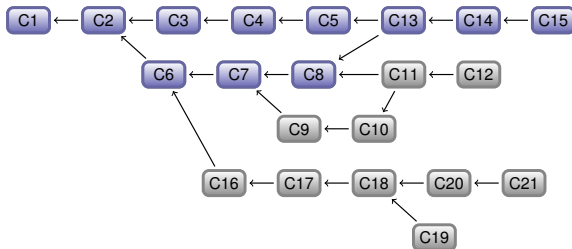
La **branche** de la version v_i d'un historique est le sous-graphe composé de l'ensemble des versions accessibles depuis v_i dans le graphe de l'historique.



Historique : Le tronc

Definition

Le **tronc** ou **branche principale** de l'historique est la branche issue de la dernière version stable.



#balanceTonDepot

Depuis le 1er octobre 2020, la branche principale des dépôts créés sur GitHub est nommée **main** au lieu de **master**...

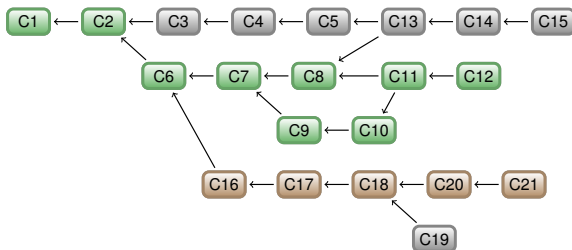
Ce changement s'inscrit dans un mouvement visant à *"supprimer les références inutiles à l'esclavage et les remplacer par des termes plus inclusifs"*.

```
$ git init
astuce: Utilisation de 'master' comme nom de la branche initiale.
astuce: Pour configurer le nom de la branche initiale pour tous les
astuce: nouveaux dépôts, et supprimer cet avertissement, lancez :
astuce:
astuce: git config --global init.defaultBranch <nom>
astuce:
astuce: Les noms les plus utilisés à la place de 'master' sont
astuce: 'main', 'trunk' et 'development'. La branche nouvellement
astuce: créée peut être renommée avec :
astuce:
astuce: git branch -m <nom>
Dépôt Git vide initialisé dans /tmp/depot
```

Historique : Sous branches

Definition

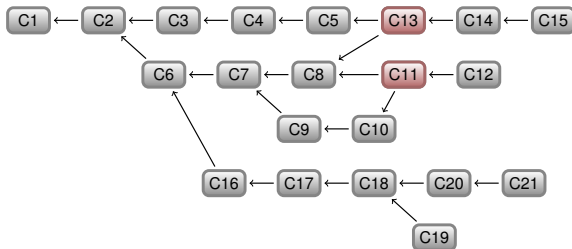
Une **sous-branche** SB_1 d'une branche B_2 est le sous graphe composé de l'ensemble des versions d'une branche B_1 n'appartenant pas à la branche B_2 , l'intersection de B_1 et de B_2 devant être non vide.



Historique : Les merges.

Definition

On appelle **merge** toute version ayant un degré sortant strictement supérieur à 1. Cette version correspond alors à la fusion des **patches** de plusieurs branches.



2001 : Linux est développé sur **CVS**.

2002 à 2005 : Linux est développé sur **Bitkeeper** :

- ▶ *Bitkeeper* est décentralisé.
- ▶ Plus *Bitkeeper* progresse et plus le développement de Linux devient efficace.

6 avril 2005 : *Bitkeeper* quitte le libre :

- ▶ Création de Git par Linus Thorvalds.

18 avril 2005 : Git peut faire un *Merge*

16 juin 2005 : Linux est développé officiellement sur **Git**.

14 février 2007 : Sortie de la version 1.5.0

- ▶ Git devient vraiment utilisable par tous !

Git c'est quoi ?

Architecture interne de git

- Création d'un dépôt

- Les objets manipulés par git

- Les "Blobs" (fichier)

- Les "Trees"

- Les "Commits"

- Les "Tags"

- Structure générale

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

Git c'est quoi ?

Architecture interne de git

- Création d'un dépôt

- Les objets manipulés par git

- Les "Blobs" (fichier)

- Les "Trees"

- Les "Commits"

- Les "Tags"

- Structure générale

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

Création d'un nouveau dépôt

La création d'un dépôt se fait avec la commande **git init**, toutes les données seront stockées dans un répertoire **.git/** à la racine du projet.

```
$ cd monProjet
$ ls -a
  Makefile README main.c
$ git init
  defaulting to local storage area
$ ls -a
.git/ Makefile README main.c
$ ls -a .git/
  HEAD branches/ config description hooks/ info/ objects/ refs/
```

Pour arrêter la gestion de version et détruire le dépôt, *i.e.*, supprimer ses données et ses métadonnées, il suffit d'effacer le répertoire **git**.

```
$ rm -rf .git/
$ ls -a
  Makefile README main.c
```

Création d'un nouveau dépôt "serveur"

Un dépôt sur un serveur n'est là que pour la gestion des versions :
il n'a donc besoin que de l'historique.

Y maintenir une version des fichiers est une source de problèmes :
on ne peut pas push dans un dépôt qui contient des fichiers modifiés

Il est donc préférable d'utiliser l'option **--bare** :

```
git init --bare ou git clone --bare <URL>
```

Ce dépôt n'a :

- ▶ pas de fichier juste l'historique
- ▶ pas de répertoire xxx/.git mais directement un xxx.git/
- ▶ pas de "remote" (origin)

```
$ git clone --bare <URL>
$ ls monProjet.git/
  HEAD branches/ config description hooks/ info/ objects/ refs/
$ cat monProjet.git/config
[core]
  bare = true
```


Git c'est quoi ?

Architecture interne de git

Création d'un dépôt

Les objets manipulés par git

Les "Blobs" (fichier)

Les "Trees"

Les "Commits"

Les "Tags"

Structure générale

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

Les objets : tout est Blob

Git a été conçu comme un : **système de fichiers versionnés**.

Linus Torvalds a repris les concepts de Linux :

- ▶ *"tout est fichier"*
→ *"tout est blob"*
- ▶ *"les fichiers sont identifiés par leur numéro d'inode"*
→ *"les blobs sont identifiés par le **SHA-1** de leur contenu"*

L'identification par *hash* n'est possible que si le contenu est fixe :
⇒ les blobs sont des objets immuables"

Cela réduit naturellement la taille des dépôts :
⇒ la déduplication niveau fichier est gratuite

Secure Hash Algorithm : SHA-1

SHA-1 est une fonction de hachage cryptographique :

- ▶ elle a été conçu par la NSA
- ▶ elle est "limitée" à des fichier de 2^{64} bits (soit 2 exaoctets)
- ▶ elle retourne un un *hash* de 160 bits
- ▶ son résultat se note avec 2+38 caractères en notation hexadécimale

```
$ echo "une petite phrase" > unFichier
$ shasum unFichier
a3bddd9c7cb436cf85a3b68ac3a653f3e9c8a40c unFichier
$ echo "Une petite phrase" > unFichier
$ shasum unFichier
e62bea749297a6f7ad5cb5744b7946dcc9036f20 unFichier
```

On peut vérifier l'intégrité d'un dépôt en recalculant tous les SHA-1

```
$ git fsck
Vérification des répertoires d'objet: 100% (256/256), fait.
```

Conséquence d'une collision de hash

Quelle que soit la fonction de hash les collisions sont inévitables :

- ▶ avec SHA-1, il y a 2^{160} fichiers en collision sur chaque hash

Conséquence d'une collision de hash

Quelle que soit la fonction de hash les collisions sont inévitables :

- ▶ avec SHA-1, il y a $2^{64}-160$ fichiers en collision sur chaque hash

Conséquence d'une collision de hash

Quelle que soit la fonction de hash les collisions sont inévitables :

- ▶ avec SHA-1, il y a $2^{64}-160$ fichiers en collision sur chaque hash

Mais dans le cas de SHA1 c'est très improbable que cela arrive :

- ▶ deux hash aléatoires ont 1 chance sur 2^{160} d'être en collision

Conséquence d'une collision de hash

Quelle que soit la fonction de hash les collisions sont inévitables :

- ▶ avec SHA-1, il y a $2^{2^{64}-160}$ fichiers en collision sur chaque hash

Mais dans le cas de SHA1 c'est très improbable que cela arrive :

- ▶ deux hash aléatoires ont 1 chance sur 2^{160} d'être en collision

Conséquence d'une collision de hash

Quelle que soit la fonction de hash les collisions sont inévitables :

- ▶ avec SHA-1, il y a $2^{64}-160$ fichiers en collision sur chaque hash

Mais dans le cas de SHA1 c'est très improbable que cela arrive :

- ▶ deux hash aléatoires ont 1 chance sur 2^{160} d'être en collision

Dans la réflexion sur le choix de la fonction de hachage, il faut distinguer :

- ▶ les conséquences d'une collision résultant de la malchance
- ▶ les risques liés à une attaque basée sur une collision de hash

Conséquence d'une collision de hash

Quelle que soit la fonction de hash les collisions sont inévitables :

- ▶ avec SHA-1, il y a $2^{64}-160$ fichiers en collision sur chaque hash

Mais dans le cas de SHA1 c'est très improbable que cela arrive :

- ▶ deux hash aléatoires ont 1 chance sur 2^{160} d'être en collision

Dans la réflexion sur le choix de la fonction de hachage, il faut distinguer :

- ▶ les conséquences d'une collision résultant de la malchance
- ▶ les risques liés à une attaque basée sur une collision de hash

Nous reviendrons sur cela une fois tous les mécanismes bien étudiés :-)

Les 4 types d'objets

Definition

Le **Blob** est l'élément de base pour le stockage des données. Il est :

- ▶ identifié par le SHA-1 de son contenu noté en hexadécimal :
`SHA-1(blob) = XXYYYYYYYYYYYYYYYYYYYYYYYYYYYYYY`
- ▶ stocké dans le dépôt sous forme d'un fichier :
`.git/objects/XX/YYYYYYYYYYYYYYYYYYYYYYYYYYYYYY`
- ▶ formé de "**<type>** **<taille du contenu>**\0**<contenu>**"
- ▶ compressé à l'aide de la bibliothèque **zlib**

Git utilise quatre types d'objets :

Blob : stocke le contenu des fichiers (*ambiguïté du Blob*)

Tree : stocke l'arborescence

Commit : stocke les versions du dépôt

Tag : identifie certaines versions du dépôt

Git c'est quoi ?

Architecture interne de git

- Création d'un dépôt

- Les objets manipulés par git

- Les "Blobs" (fichier)

- Les "Trees"

- Les "Commits"

- Les "Tags"

- Structure générale

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

Les "Blobs" (fichiers)

Definition

On appelle aussi **Blob**, un blob qui stocke le contenu d'un fichier :

- ▶ ils sont composés de :
`"blob <taille du contenu>\0<contenu>"`
- ▶ à chaque révision du fichier correspond un nouveau *Blob*.
- ▶ le *Blob* ne dépend pas du nom ou de l'emplacement :
 - ▶ Si un fichier est renommé, pas de nouveau *Blob*
 - ▶ Si un fichier est déplacé, pas de nouveau *Blob*

Les "Blobs" (fichiers) : exemple

33f3a..

Blob	Size
Mouches d'automne Comme il est facile De vous écrabouiller! <i>Shiki (1866 - 1902)</i>	

a35e1..

Blob	Size
Mante religieuse Piteuse escaladeuse De ce melon <i>Shiki (1866 - 1902)</i>	

Les "Blobs" sont compressés avec la zlib

```
$ cat foo.txt  
Mouches d'automne  
Comme il est facile  
De vous écrabouiller !  
  
Shiki ( 1866 - 1902 )
```

Les "Blobs" sont compressés avec la zlib

```
$ cat foo.txt
Mouches d'automne
Comme il est facile
De vous écrabouiller !

Shiki ( 1866 - 1902 )

$ git cat-file -p 6ded47ba437cb9cc986e47bafc146c75e50be03c
Mouches d'automne
Comme il est facile
De vous écrabouiller !

Shiki ( 1866 - 1902 )
```

Les "Blobs" sont compressés avec la zlib

```
$ cat foo.txt  
Mouches d'automne  
Comme il est facile  
De vous écrabouiller !
```

```
Shiki ( 1866 - 1902 )
```

```
$ git cat-file -p 6ded47ba437cb9cc986e47bafc146c75e50be03c  
Mouches d'automne  
Comme il est facile  
De vous écrabouiller !
```

```
Shiki ( 1866 - 1902 )
```

```
$ cat ../git/objects/6d/ed47ba437cb9cc986e47bafc146c75e50be03c  
xKOR0e/MH-VHQO,-KrMUQH-.QHKLIrIU(/-V82(1)43' 'HA+8#3;SACLAWHA
```


Les "Blobs" sont compressés avec la zlib

```
$ cat foo.txt
Mouches d'automne
Comme il est facile
De vous écrabouiller !
```

```
Shiki ( 1866 - 1902 )
```

```
$ git cat-file -p 6ded47ba437cb9cc986e47bafc146c75e50be03c
Mouches d'automne
Comme il est facile
De vous écrabouiller !
```

```
Shiki ( 1866 - 1902 )
```

```
$ cat ../git/objects/6d/ed47ba437cb9cc986e47bafc146c75e50be03c
xKOR0e/MH-VHQO,-KrMUQH-.QHKLirIU(/-V82(1)43' 'HA+8#3;SACLAWHA
```

```
$ pigz --decompress --zlib < ../git/objects/6d/ed47ba437cb9cc9*
blob 85Mouches d'automne
Comme il est facile
De vous écrabouiller !
```

```
Shiki ( 1866 - 1902 )
```

Le SHA-1 est calculé sur le blob pas sur le contenu

```
$ sha1sum foo.txt  
df8ac3abf89ac0545a35783ca23c138408a0c476 foo.txt
```

Le SHA-1 est calculé sur le blob pas sur le contenu

```
$ shasum foo.txt
df8ac3abf89ac0545a35783ca23c138408a0c476 foo.txt

$ ls .git/objects/df/*
/usr/bin/ls: impossible d'accéder à '.git/objects/df/*'
```

Le SHA-1 est calculé sur le blob pas sur le contenu

```
$ shasum foo.txt
df8ac3abf89ac0545a35783ca23c138408a0c476 foo.txt

$ ls .git/objects/df/*
/usr/bin/ls: impossible d'accéder à '.git/objects/df/*'

$ echo -en "blob $(wc -c < foo.txt)\0" | cat - foo.txt | shasum
6ded47ba437cb9cc986e47bafc146c75e50be03c -
```

Le SHA-1 est calculé sur le blob pas sur le contenu

```
$ shasum foo.txt
df8ac3abf89ac0545a35783ca23c138408a0c476 foo.txt

$ ls .git/objects/df/*
/usr/bin/ls: impossible d'accéder à '.git/objects/df/*'

$ echo -en "blob $(wc -c < foo.txt)\0" | cat - foo.txt | shasum
6ded47ba437cb9cc986e47bafc146c75e50be03c -

$ ls .git/objects/6d/*
.git/objects/6d/ed47ba437cb9cc986e47bafc146c75e50be03c
```

Le SHA-1 est calculé sur le blob pas sur le contenu

```
$ shasum foo.txt
df8ac3abf89ac0545a35783ca23c138408a0c476 foo.txt

$ ls .git/objects/df/*
/usr/bin/ls: impossible d'accéder à '.git/objects/df/*'

$ echo -en "blob $(wc -c < foo.txt)\0" | cat - foo.txt | shasum
6ded47ba437cb9cc986e47bafc146c75e50be03c -

$ ls .git/objects/6d/*
.git/objects/6d/ed47ba437cb9cc986e47bafc146c75e50be03c

$ git hash-object foo.txt
6ded47ba437cb9cc986e47bafc146c75e50be03c
```

Git c'est quoi ?

Architecture interne de git

- Création d'un dépôt

- Les objets manipulés par git

- Les "Blobs" (fichier)

- Les "Trees"

- Les "Commits"

- Les "Tags"

- Structure générale

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

Definition

Un **Tree** stocke la liste des fichiers d'un répertoire :

- ▶ Un **Tree** est un ensemble de pointeurs vers des **Blobs** et d'autres **Trees**.
- ▶ Un **Tree** associe un nom de fichier (resp. repertoire) à chacun des pointeurs de **Blobs** (resp. **Trees**).
- ▶ Un ensemble de **Tree** permet de décrire l'état d'une hiérarchie de dossiers à un moment donné.

Les "Tree" : exemple

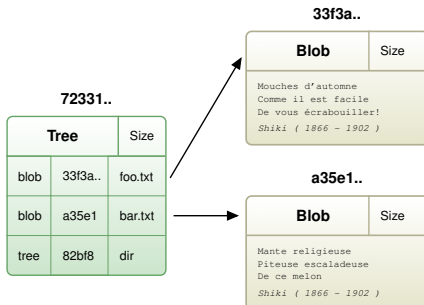
33f3a..

Blob	Size
Mouches d'automne Comme il est facile De vous écrabouiller! <i>Shiki (1866 - 1902)</i>	

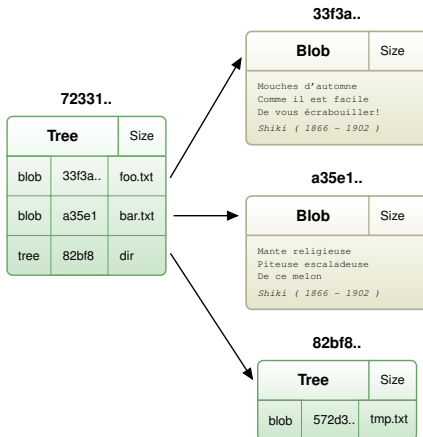
a35e1..

Blob	Size
Mante religieuse Piteuse escaladeuse De ce melon <i>Shiki (1866 - 1902)</i>	

Les "Tree" : exemple



Les "Tree" : exemple



Stockage de la hierarchie

```
$ ls -R
.:
bar.txt dir/ foo.txt

./dir:
tmp.txt
```

Stockage de la hierarchie

```
$ ls -R
.:
bar.txt dir/ foo.txt

./dir:
tmp.txt

$ git cat-file -p e7f3b24b01115fd0ea58d72099a24af383f627ba
100644 blob b13fb47577e28a3069183f9ad91fb0ee44d3871f bar.txt
040000 tree 96a31457a25444a576b1fb0b8eafee735a706693 dir
100644 blob 6ded47ba437cb9cc986e47bafc146c75e50be03c foo.txt
```

Stockage de la hierarchie

```
$ ls -R
.:
bar.txt dir/ foo.txt

./dir:
tmp.txt

$ git cat-file -p e7f3b24b01115fd0ea58d72099a24af383f627ba
100644 blob b13fb47577e28a3069183f9ad91fb0ee44d3871f bar.txt
040000 tree 96a31457a25444a576b1fb0b8eafee735a706693 dir
100644 blob 6ded47ba437cb9cc986e47bafc146c75e50be03c foo.txt

$ git cat-file -p 96a31
100644 blob e69de29bb2d1d6434b8b29ae775ad8c2e48c5391 tmp.txt
```

Git c'est quoi ?

Architecture interne de git

- Création d'un dépôt

- Les objets manipulés par git

- Les "Blobs" (fichier)

- Les "Trees"

- Les "Commits"

- Les "Tags"

- Structure générale

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

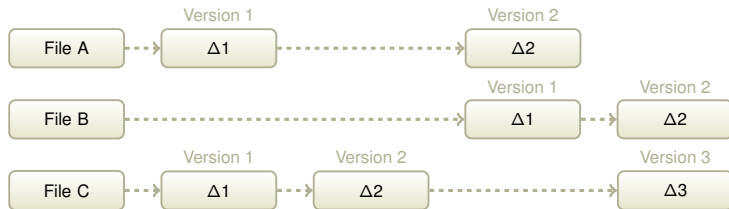
Commit : définition.

Definition

Committer un fichier signifie : enregistrer une version de ce dernier (le plus souvent la version actuelle, mais pas toujours) dans un gestionnaire de versions. Par extension, si le fichier est déjà versionné, on dit que l'on **commit** une modification du fichier.

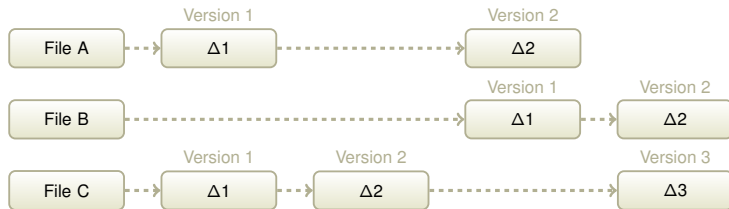
La numérotation des commits dans *CVS*.

CVS utilise une numérotation sur des patches, fichier par fichier :



La numérotation des commits dans CVS.

CVS utilise une numérotation sur des patches, fichier par fichier :
=> difficile de trouver un état cohérent du système (ajout de tag).

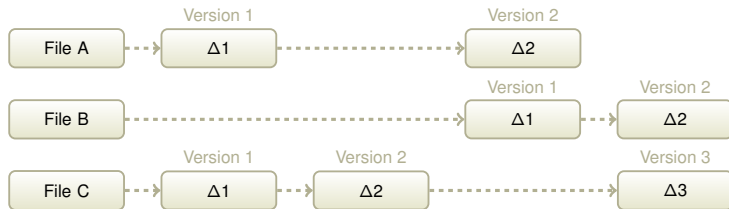


La numérotation des commits dans CVS.

CVS utilise une numérotation sur des patches, fichier par fichier :

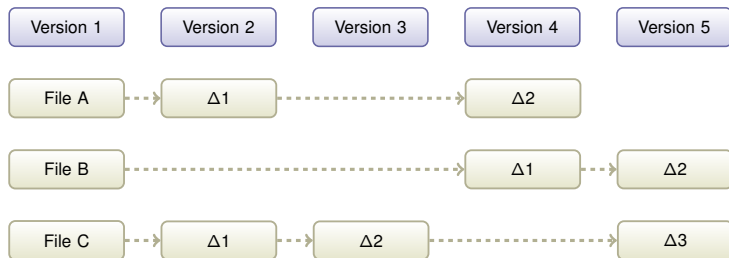
=> difficile de trouver un état cohérent du système (ajout de tag).

=> l'accès à une version nécessite de ré-appliquer les patches.



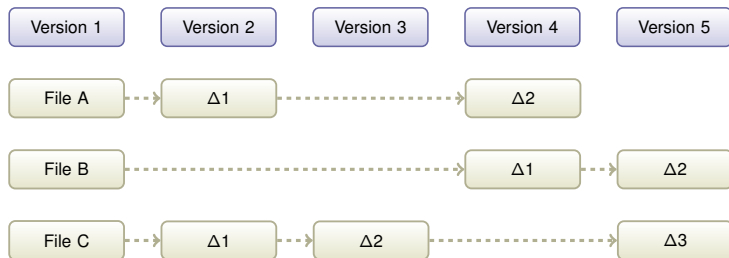
La numérotation des commits dans *SVN*.

SVN utilise une numérotation globale sur l'ensemble des patches :



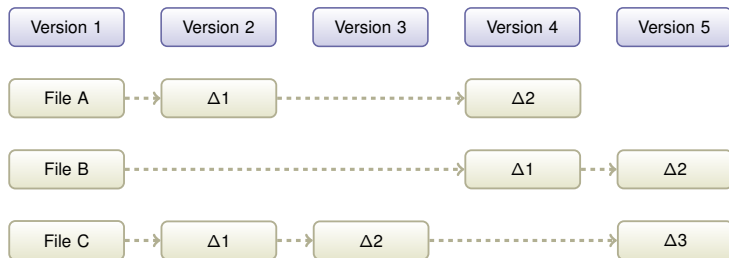
La numérotation des commits dans *SVN*.

SVN utilise une numérotation globale sur l'ensemble des patches :
=> chaque numéro de version correspond à un état cohérent ;



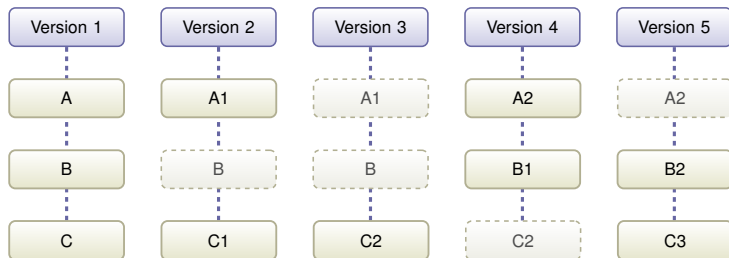
La numérotation des commits dans *SVN*.

- SVN utilise une numérotation globale sur l'ensemble des patches :
- => chaque numéro de version correspond à un état cohérent ;
 - => l'accès à une version nécessite de ré-appliquer les patches.



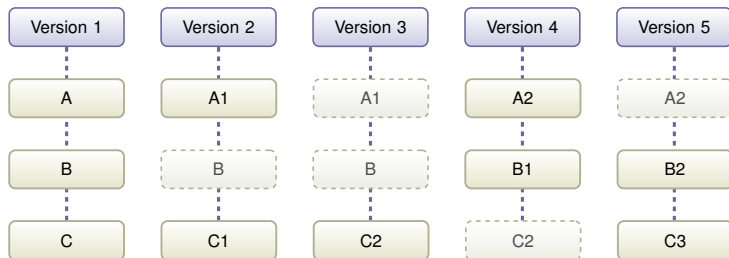
La numérotation des commits dans *Git*.

Git utilise une numérotation globale des versions des fichiers :



La numérotation des commits dans *Git*.

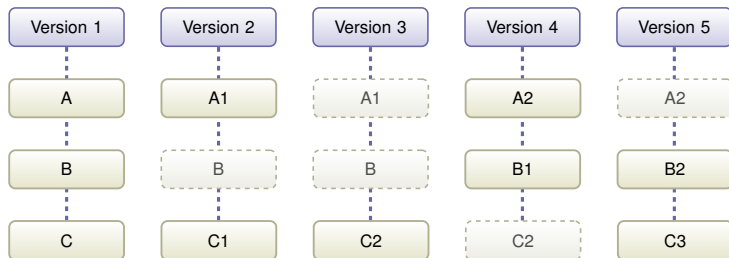
Git utilise une numérotation globale des versions des fichiers :
=> chaque numéro de version correspond à un état cohérent ;



La numérotation des commits dans *Git*.

Git utilise une numérotation globale des versions des fichiers :

- => chaque numéro de version correspond à un état cohérent ;
- => accès direct aux versions du système.



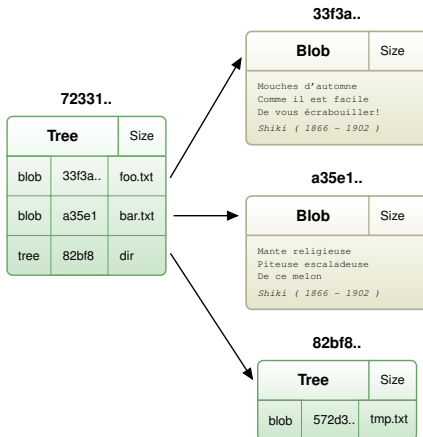
Les "Commits" : définitions

Definition

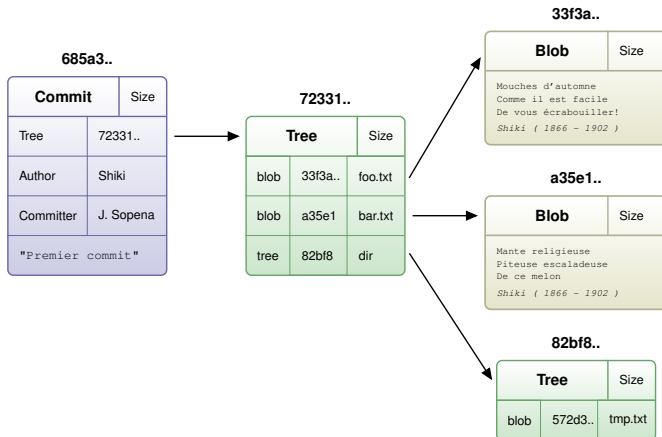
Un **Commit** stocke l'état d'une partie du dépôt à un instant donné. Il contient :

- ▶ un pointeur vers un **Tree** dont on souhaite sauver l'état.
- ▶ un pointeur vers un ou plusieurs autres **Commits** pour constituer un historique.
- ▶ le nom d'un **auteur** et d'un **commiteur**.
- ▶ une description sous forme d'une chaîne de caractères.

Les "Commits" : exemple



Les "Commits" : exemple



Contenu des blobs commit

Attention : les blobs *commit* contiennent des références à la date (en notation Epoch). Contrairement aux autres blobs, vous n'obtiendrez pas les même hash si vous refaites l'expérience.

```
$ pigz --decompress --zlib < .git/objects/10/5a55d3153a65563393*  
commit 175tree e7f3b24b01115fd0ea58d72099a24af383f627ba  
author Shiki <shiki@haikus.jp> 1633967030 +0200  
committer Julien SOPENA <julien.sopena@lip6.fr> 1633967030 +0200
```

Premier commit

Contenu des blobs commit

Attention : les blobs *commit* contiennent des références à la date (en notation Epoch). Contrairement aux autres blobs, vous n'obtiendrez pas les même hash si vous refaites l'expérience.

```
$ pigz --decompress --zlib < .git/objects/10/5a55d3153a65563393*
commit 175tree e7f3b24b01115fd0ea58d72099a24af383f627ba
author Shiki <shiki@haikus.jp> 1633967030 +0200
committer Julien SOPENA <julien.sopena@lip6.fr> 1633967030 +0200
```

Premier commit

```
$ git cat-file -p e7f3b24b01115fd0ea58d72099a24af383f627ba
100644 blob b13fb47577e28a3069183f9ad91fb0ee44d3871f bar.txt
040000 tree 96a31457a25444a576b1fb0b8eafee735a706693 dir
100644 blob 6ded47ba437cb9cc986e47bafc146c75e50be03c foo.txt
```

Contenu des blobs commit

Attention : les blobs *commit* contiennent des références à la date (en notation Epoch). Contrairement aux autres blobs, vous n'obtiendrez pas les même hash si vous refaites l'expérience.

```
$ pigz --decompress --zlib < .git/objects/10/5a55d3153a65563393*
commit 175tree e7f3b24b01115fd0ea58d72099a24af383f627ba
author Shiki <shiki@haikus.jp> 1633967030 +0200
committer Julien SOPENA <julien.sopena@lip6.fr> 1633967030 +0200
```

Premier commit

```
$ git cat-file -p e7f3b24b01115fd0ea58d72099a24af383f627ba
100644 blob b13fb47577e28a3069183f9ad91fb0ee44d3871f bar.txt
040000 tree 96a31457a25444a576b1fb0b8eafee735a706693 dir
100644 blob 6ded47ba437cb9cc986e47bafc146c75e50be03c foo.txt
```

La suite un peu plus tard

Git c'est quoi ?

Architecture interne de git

- Création d'un dépôt

- Les objets manipulés par git

- Les "Blobs" (fichier)

- Les "Trees"

- Les "Commits"

- Les "Tags"

- Structure générale

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

Definition

Un **tag** permet d'identifier un des objets précédents à l'aide d'un *nom*. Il contient :

- ▶ un pointeur vers un **Blob**, un **Tree** ou un **Commit**.
- ▶ une signature.

Commandes basiques du dépôt

init : initialisation d'un dépôt.

clone : copie d'un dépôt existant (local ou distant).

fsck-object : pour valider un dépôt.

repack : fait des paquets de **blobs** pour l'efficacité.

prune : supprime les **blobs** uniques mais existants dans un paquet.

Git c'est quoi ?

Architecture interne de git

- Création d'un dépôt

- Les objets manipulés par git

- Les "Blobs" (fichier)

- Les "Trees"

- Les "Commits"

- Les "Tags"

- Structure générale

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

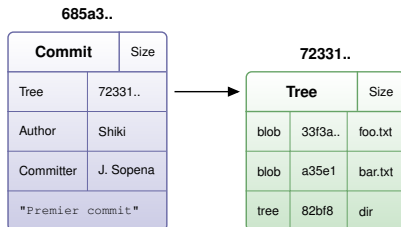
Conclusion

Graphe acyclique des objets.

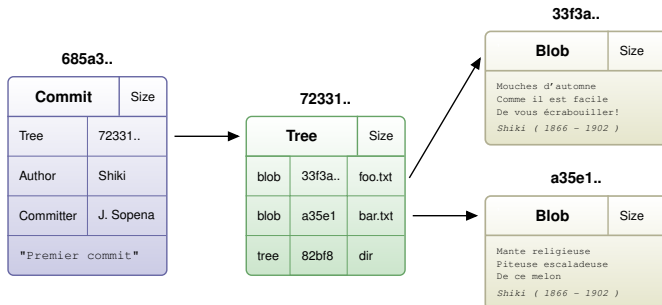
685a3..

Commit		Size
Tree	72331..	
Author	Shiki	
Committer	J. Sopena	
"Premier commit"		

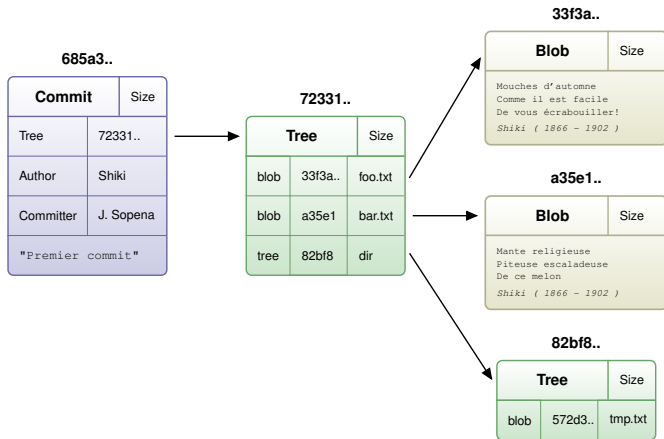
Graphe acyclique des objets.



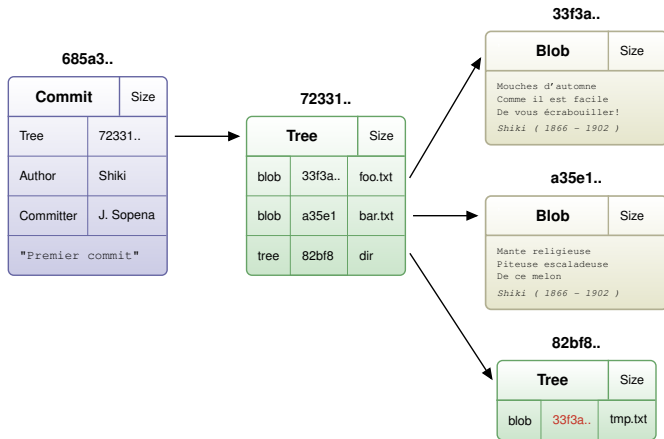
Graphe acyclique des objets.



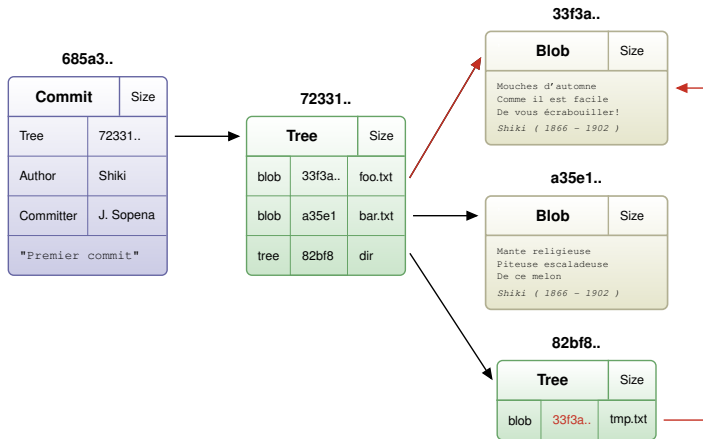
Graphe acyclique des objets.



Déduplication implicite



Déduplication implicite



Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

- Les commits

- Les branches

- Les merges

- Les rebases

- Les remords

- Utilisation de l'historique

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

Definition

Git est un ensemble de commandes indépendantes permettant d'archiver, de rechercher et de publier des ensembles d'objets représentant l'état global de l'espace de travail versionné à un instant donné.

Toutes ces commandes :

- ▶ portent un nom du type **git-`<nom_de_la_commande>`**.
- ▶ ont un équivalent sous la forme d'une option de la commande **git `<nom_de_la_commande>`**.

Exemple

```
git-add monFichier.txt  $\iff$  git add  
monFichier.txt
```

Git c'est un ensemble de 145 commandes.

add	daemon	index-pack	parse-remote	shortlog
add-	describe	init	patch-id	show
interactive	diff	instaweb	peek-remote	show-branch
am	diff-files	log	prune	show-index
annotate	diff-index	lost-found	prune-packed	show-ref
apply	diff-tree	ls-files	pull	stage
archimport	difftool	ls-remote	push	stash
archive	difftool-	ls-tree	quiltimport	status
bisect	helper	mailinfo	read-tree	strip-space
bisect-helper	fast-export	mailsplit	rebase	submodule
blame	fast-import	merge	rebase-	svn
branch	fetch	merge-base	interactive	symbolic-ref
bundle	fetch-tool	merge-file	receive-pack	tag
cat-file	fetch-pack	merge-index	reflog	tar-tree
check-attr	filter-branch	merge-octopus	relink	unpack-file
check-ref-	fmt-merge-msg	merge-one-file	remote	unpack-objects
format	for-each-ref	merge-ours	remote-curl	update-index
checkout	format-patch	merge-	repack	update-ref
checkout-index	fsck	recursive	replace	update-server-
cherry	fsck-objects	merge-resolve	repo-config	info
cherry-pick	gc	merge-subtree	request-pull	upload-archive
citool	get-tar-	merge-tree	rerere	upload-pack
clean	commit-id	mergetool	reset	var
clone	grep	mergetool-lib	rev-list	verify-pack
commit	gui (et gitk)	mktag	rev-parse	verify-tag
commit-tree	gui-askpass	mktree	revert	web-browse
config	hash-object	mv	rm	whatchanged
count-objects	help	name-rev	send-email	write-tree
cvsexportcommit	http-fetch	pack-objects	send-pack	
cvsimpport	http-push	pack-redundant	sh-setup	
cvsserver	imap-send	pack-refs	shell	

Git c'est un ensemble de 145 commandes.

Les commandes permettant de gérer un dépôt local

add
init
diff
log
prune
stash
status
rebase
tag
checkout
reset
revert
rm
mv
help
config
commit
grep
branch

Git c'est un ensemble de 145 commandes.

Les commandes permettant gérer des dépôts distants

add

am

branch

checkout

clone

commit

config

diff

fetch

grep

help

init

log

merge

mv

prune

pull

push

rebase

reset

revert

rm

stash

status

tag

Git c'est un ensemble de 145 commandes.

git-gui et gitk pour l'interface graphique.

add
am
branch
checkout
clone
commit
config
diff
fetch
grep
gui (et gitk)
help
init
log
merge
mv
prune
pull
push
rebase
reset
revert
rm
stash
status
tag

Git c'est un ensemble de 145 commandes.

Les commandes pour une utilisation avancée

add
init
am
diff
log
prune
bisect
pull
push
stash
status
blame
merge
rebase
branch
fetch
tag
checkout
repack
gc
rerere
reset
clone
grep
gui (et gitk)
revert
rm
config
mv
help

Git c'est un ensemble de 145 commandes.

Les autres commandes au cas par cas

add	cvsserver	index-pack	pack-refs	shortlog
add-	daemon	init	parse-remote	show
interactive	describe	instaweb	patch-id	show-branch
am	diff	log	peek-remote	show-index
annotate	diff-files	lost-found	prune	show-ref
apply	diff-index	ls-files	prune-packed	stage
archimport	diff-tree	ls-remote	pull	stash
archive	difftool	ls-tree	push	status
bisect	difftool-	mailinfo	quiltimport	strip-space
bisect-helper	helper	mailsplit	read-tree	submodule
blame	fast-export	merge	rebase	svn
branch	fast-import	merge-base	rebase-	symbolic-ref
bundle	fetch	merge-file	interactive	tag
cat-file	fetch-tool	merge-index	receive-pack	tar-tree
check-attr	fetch-pack	merge-octopus	reflog	unpack-file
check-ref-	filter-branch		relink	
format	fmt-merge-msg	merge-one-file	remote	unpack-objects
checkout	for-each-ref	merge-ours	remote-curl	update-index
	format-patch	merge-	repack	update-ref
checkout-index	fsck	recursive	replace	update-
cherry	fsck-objects	merge-resolve	repo-config	server-info
cherry-pick	gc	merge-subtree	request-pull	
citool	get-tar-	merge-tree	rerere	upload-archive
clean	commit-id	mergetool	reset	upload-pack
clone	grep	mergetool-lib	rev-list	var
commit	gui (et gitk)	mktag	rev-parse	verify-pack
commit-tree	gui-askpass	mktree	revert	verify-tag
config	hash-object	mv	rm	web-browse
count-objects	help	name-rev	send-email	what-changed
cvsexportcom-	http-fetch	pack-objects	send-pack	write-tree
mit	http-push		sh-setup	
cvimport	imap-send	pack-redundant	shell	

Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

- Les commits

- Les branches

- Les merges

- Les rebases

- Les remords

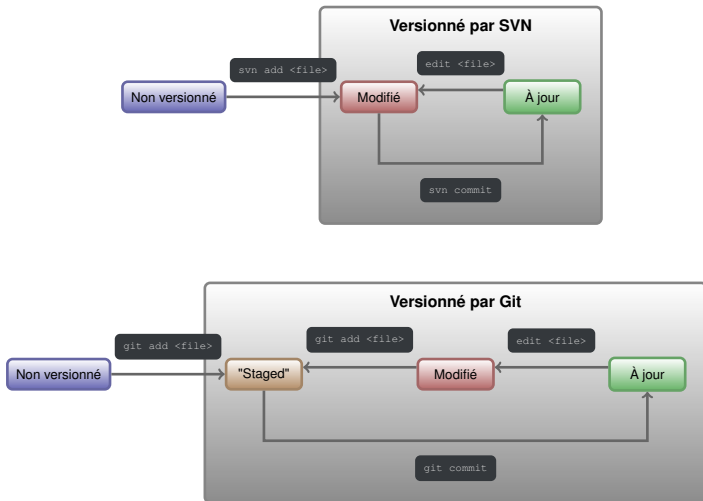
- Utilisation de l'historique

Synchronisation avec les dépôts distants.

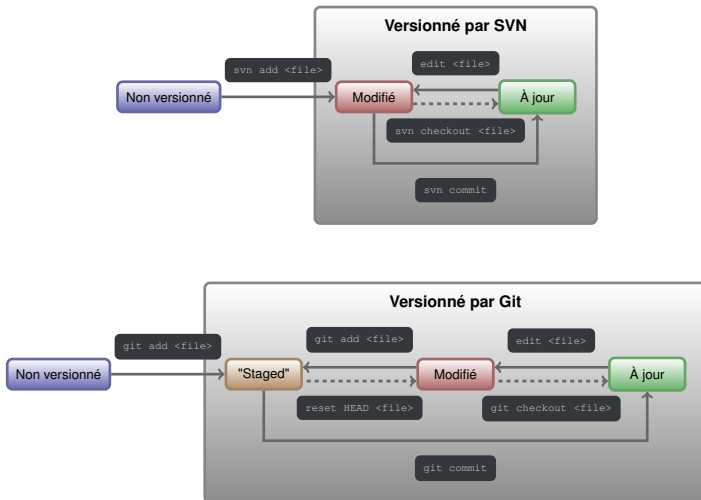
Les outils graphiques

Conclusion

Commit : différence avec cvs/svn.



Commit : différence avec cvs/svn.



Commit : les commandes.

add : ajoute dans l'index un fichier à commiter **dans son état actuel**.

commit : enregistre dans le dépôt local les modifications qui ont été **ajoutées dans l'index** par une commande **add**.

reset HEAD : supprime la référence d'un fichier de l'index ajouté par une commande **add**.

Exemple

```
$ echo "Premier fichier" > foo.txt
$ git add foo.txt
$ git commit -m "Description de ce commit"
```

Étude des objets générés par un exemple simple.

```
$ mkdir project  
$ cd project  
$ git init
```

Étude des objets générés par un exemple simple.

```
$ mkdir project  
$ cd project  
$ git init
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

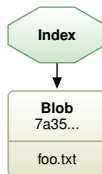
$ echo "toto" > foo.txt
$ git add foo.txt
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

$ git commit -m "Add foo.txt"
```

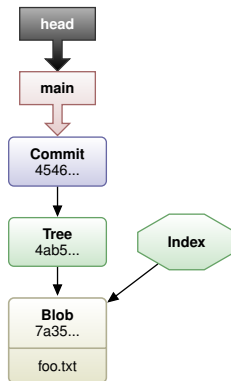


Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

$ git commit -m "Add foo.txt"
```



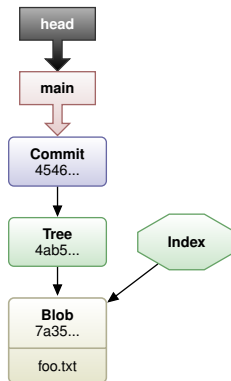
Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt
```



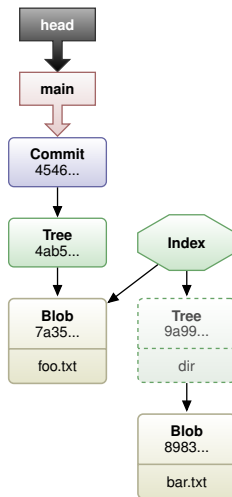
Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt
```



Étude des objets générés par un exemple simple.

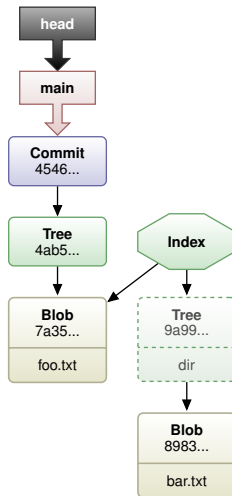
```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Add dir/bar.txt"
```



Étude des objets générés par un exemple simple.

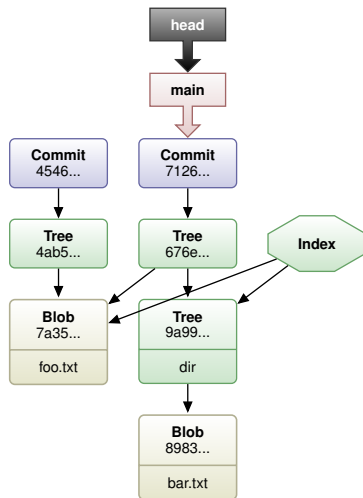
```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Add dir/bar.txt"
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

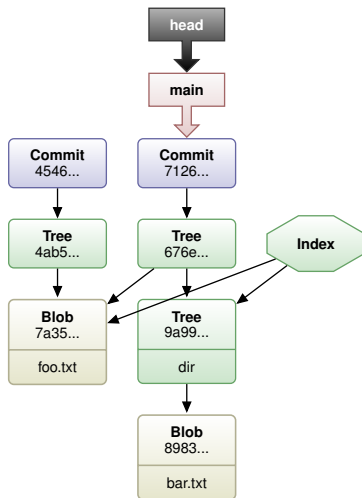
$ echo "toto" > foo.txt
$ git add foo.txt

$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Add dir/bar.txt"

$ echo "tutu" >> dir/bar.txt
$ git add dir/bar.txt
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

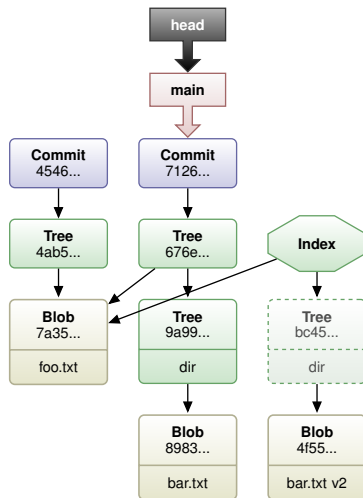
$ echo "toto" > foo.txt
$ git add foo.txt

$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Add dir/bar.txt"

$ echo "tutu" >> dir/bar.txt
$ git add dir/bar.txt
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

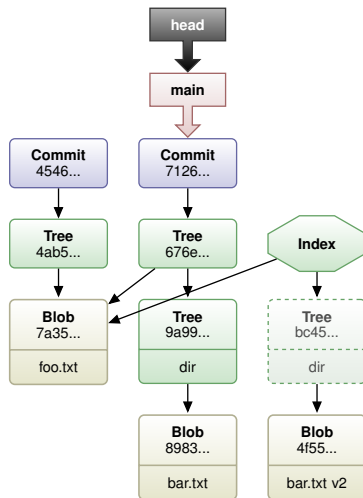
$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Add dir/bar.txt"

$ echo "tutu" >> dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Modif dir/bar.txt"
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

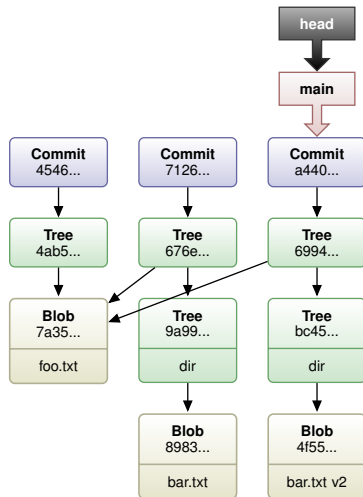
$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Add dir/bar.txt"

$ echo "tutu" >> dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Modif dir/bar.txt"
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Add dir/bar.txt"

$ echo "tutu" >> dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Modif dir/bar.txt"
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

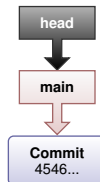
$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Add dir/bar.txt"

$ echo "tutu" >> dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Modif dir/bar.txt"
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

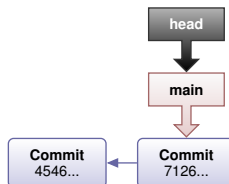
$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Add dir/bar.txt"

$ echo "tutu" >> dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Modif dir/bar.txt"
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

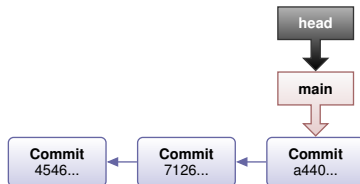
$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Add dir/bar.txt"

$ echo "tutu" >> dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Modif dir/bar.txt"
```



Étude des objets générés par un exemple simple.

```
$ mkdir project
$ cd project
$ git init

$ echo "toto" > foo.txt
$ git add foo.txt

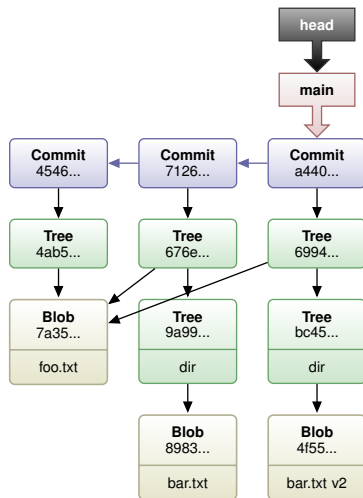
$ git commit -m "Add foo.txt"

$ mkdir dir
$ echo "titi" > dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Add dir/bar.txt"

$ echo "tutu" >> dir/bar.txt
$ git add dir/bar.txt

$ git commit -m "Modif dir/bar.txt"
```



Chaîne de commit dans les Blobs

```
$ pigz --decompress --zlib < .git/objects/10/5a55d3153a65563393*  
commit 175tree e7f3b24b01115fd0ea58d72099a24af383f627ba  
author Shiki <shiki@haikus.jp> 1633967030 +0200  
committer Julien SOPENA <julien.sopena@lip6.fr> 1633967030 +0200
```

Premier commit

Chaîne de commit dans les Blobs

```
$ pigz --decompress --zlib < .git/objects/10/5a55d3153a65563393*  
commit 175tree e7f3b24b01115fd0ea58d72099a24af383f627ba  
author Shiki <shiki@haikus.jp> 1633967030 +0200  
committer Julien SOPENA <julien.sopena@lip6.fr> 1633967030 +0200
```

Premier commit

```
$ TODO
```

Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

- Les commits

- Les branches

- Les merges

- Les rebases

- Les remords

- Utilisation de l'historique

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

Branche : les commandes.

branch : liste les branches avec une ***** pour la branche active.

branch <nom> : crée une nouvelle branche <nom> ;

branch -m : permet de renommer une branche ;

branch -d : permet de supprimer une branche ;

switch : change de branche active ;

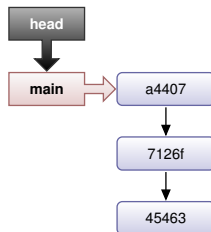
switch -c : crée une branche et l'active ;

show-branch : affiche les branches et leurs commits.

```
$ git branch
* main
$ git branch maBranche
$ git branch
maBranche
* main
$ git switch maBranche
$ git branch
* maBranche
main
```

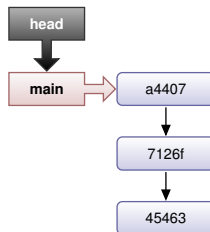
Branche : isoler provisoirement un développement

```
$ ls  
foo.txt dir
```



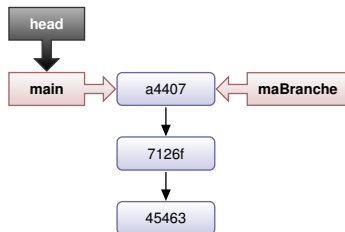
Branche : isoler provisoirement un développement

```
$ ls  
foo.txt dir  
  
$ git branch maBranche
```



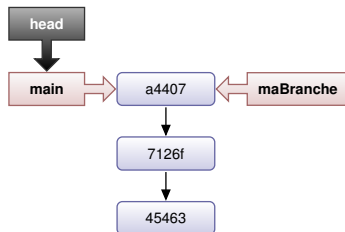
Branche : isoler provisoirement un développement

```
$ ls  
foo.txt dir  
  
$ git branch maBranche
```



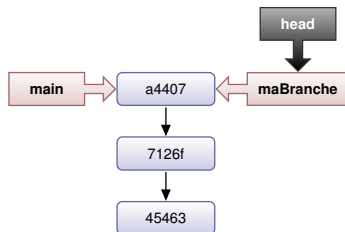
Branche : isoler provisoirement un développement

```
$ ls  
foo.txt dir  
  
$ git branch maBranche  
  
$ git switch maBranche
```



Branche : isoler provisoirement un développement

```
$ ls  
foo.txt dir  
  
$ git branch maBranche  
  
$ git switch maBranche
```



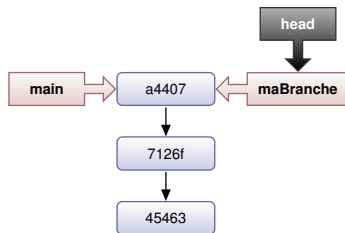
Branche : isoler provisoirement un développement

```
$ ls
foo.txt dir

$ git branch maBranche

$ git switch maBranche

$ touch fichier1.txt
$ ls
dir fichier1.txt foo.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"
```



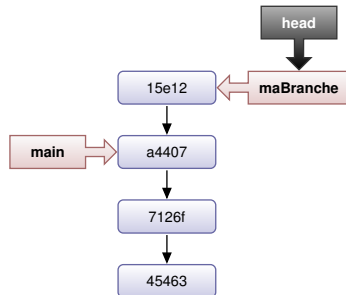
Branche : isoler provisoirement un développement

```
$ ls
foo.txt dir

$ git branch maBranche

$ git switch maBranche

$ touch fichier1.txt
$ ls
dir fichier1.txt foo.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"
```



Branche : isoler provisoirement un développement

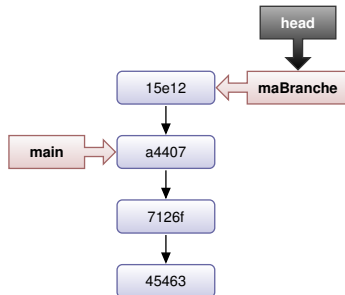
```
$ ls
foo.txt dir

$ git branch maBranche

$ git switch maBranche

$ touch fichier1.txt
$ ls
dir fichier1.txt foo.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ git switch main
$ ls
dir foo.txt
```



Branche : isoler provisoirement un développement

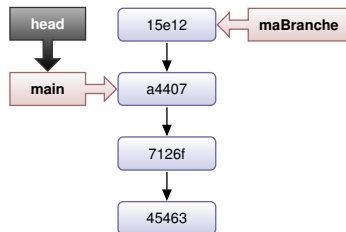
```
$ ls
foo.txt dir

$ git branch maBranche

$ git switch maBranche

$ touch fichier1.txt
$ ls
dir fichier1.txt foo.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ git switch main
$ ls
dir foo.txt
```



Branche : isoler provisoirement un développement

```
$ ls
foo.txt dir

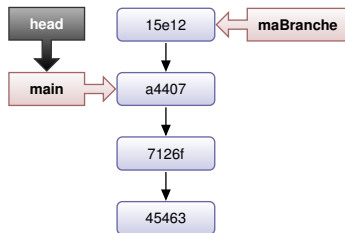
$ git branch maBranche

$ git switch maBranche

$ touch fichier1.txt
$ ls
dir fichier1.txt foo.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ git switch main
$ ls
dir foo.txt

$ touch fichier2.txt
$ git add fichier2.txt
$ git commit -m "Add fichier2.txt"
```



Branche : isoler provisoirement un développement

```
$ ls
foo.txt dir

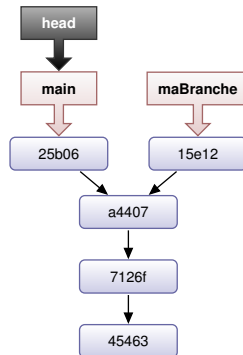
$ git branch maBranche

$ git switch maBranche

$ touch fichier1.txt
$ ls
dir fichier1.txt foo.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ git switch main
$ ls
dir foo.txt

$ touch fichier2.txt
$ git add fichier2.txt
$ git commit -m "Add fichier2.txt"
```



Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

- Les commits

- Les branches

- Les merges

- Les rebases

- Les remords

- Utilisation de l'historique

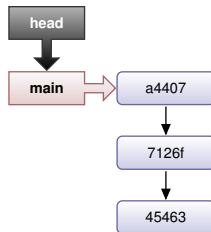
Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

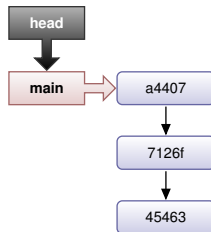
fast-forward : merge sans création de commit

```
$ ls  
foo.txt dir
```



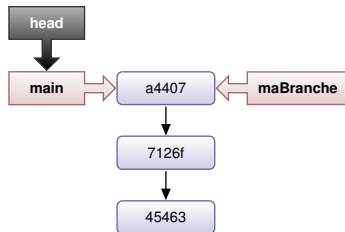
fast-forward : merge sans création de commit

```
$ ls  
foo.txt dir  
  
$ git switch -c maBranche
```



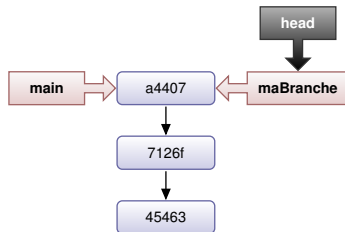
fast-forward : merge sans création de commit

```
$ ls  
foo.txt dir  
  
$ git switch -c maBranche
```



fast-forward : merge sans création de commit

```
$ ls  
foo.txt dir  
  
$ git switch -c maBranche
```

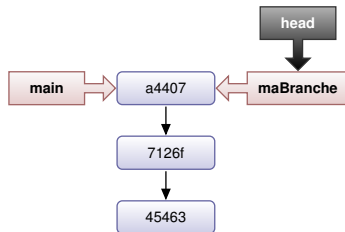


fast-forward : merge sans création de commit

```
$ ls
foo.txt dir

$ git switch -c maBranche

$ echo "toto" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"
```

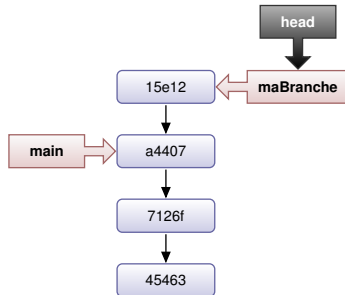


fast-forward : merge sans création de commit

```
$ ls
foo.txt dir

$ git switch -c maBranche

$ echo "toto" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"
```



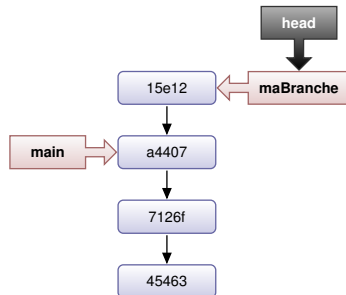
fast-forward : merge sans création de commit

```
$ ls
foo.txt dir

$ git switch -c maBranche

$ echo "toto" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"
```



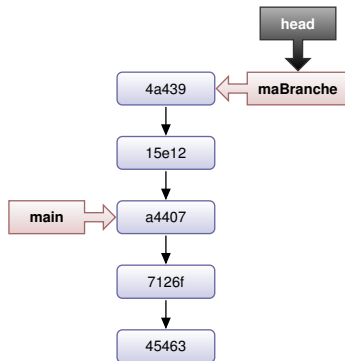
fast-forward : merge sans création de commit

```
$ ls
foo.txt dir

$ git switch -c maBranche

$ echo "toto" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"
```



fast-forward : merge sans création de commit

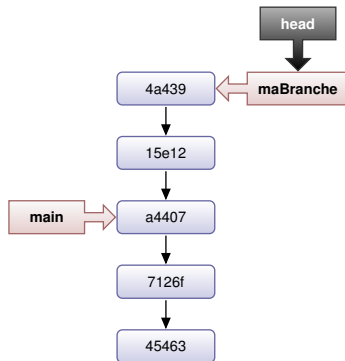
```
$ ls
foo.txt dir

$ git switch -c maBranche

$ echo "toto" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main
$ ls
dir foo.txt
```



fast-forward : merge sans création de commit

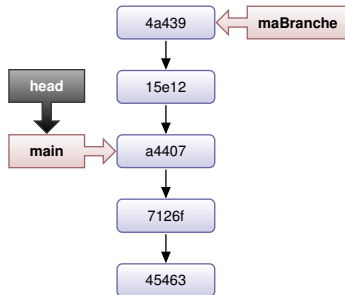
```
$ ls
foo.txt dir

$ git switch -c maBranche

$ echo "toto" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main
$ ls
dir foo.txt
```



fast-forward : merge sans création de commit

```
$ ls
foo.txt dir

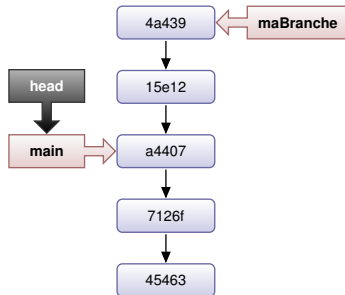
$ git switch -c maBranche

$ echo "toto" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main
$ ls
dir foo.txt

$ git merge maBranche
$ cat fichier1.txt
titi
```



fast-forward : merge sans création de commit

```
$ ls
foo.txt dir

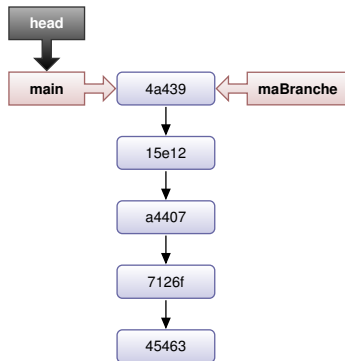
$ git switch -c maBranche

$ echo "toto" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main
$ ls
dir foo.txt

$ git merge maBranche
$ cat fichier1.txt
titi
```



fast-forward : merge sans création de commit

```
$ ls
foo.txt dir

$ git switch -c maBranche

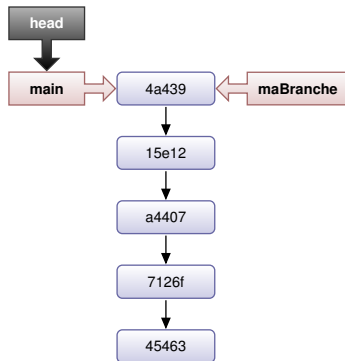
$ echo "toto" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main
$ ls
dir foo.txt

$ git merge maBranche
$ cat fichier1.txt
titi

$ git branch -d maBranche
```



fast-forward : merge sans création de commit

```
$ ls
foo.txt dir

$ git switch -c maBranche

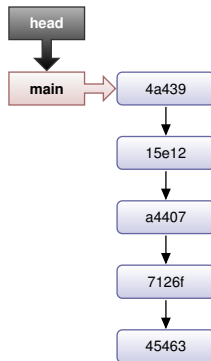
$ echo "toto" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main
$ ls
dir foo.txt

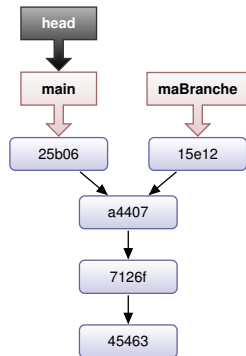
$ git merge maBranche
$ cat fichier1.txt
titi

$ git branch -d maBranche
```



Merge : exemple sur deux branches distinctes.

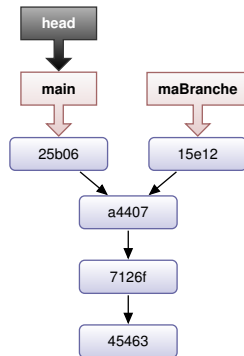
```
$ ls  
dir fichier2.txt foo.txt
```



Merge : exemple sur deux branches distinctes.

```
$ ls
dir fichier2.txt foo.txt

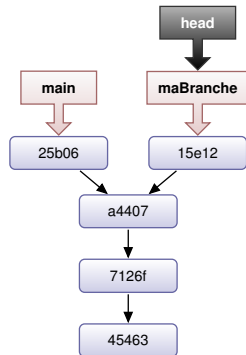
$ git switch maBranche
$ ls
dir fichier1.txt foo.txt
```



Merge : exemple sur deux branches distinctes.

```
$ ls
dir fichier2.txt foo.txt

$ git switch maBranche
$ ls
dir fichier1.txt foo.txt
```

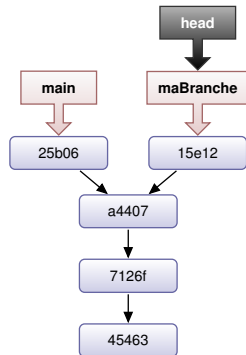


Merge : exemple sur deux branches distinctes.

```
$ ls
dir fichier2.txt foo.txt

$ git switch maBranche
$ ls
dir fichier1.txt foo.txt

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"
```

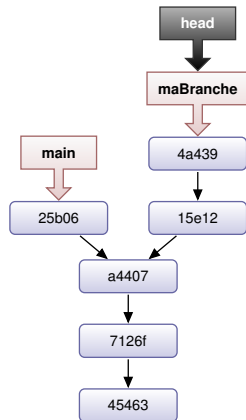


Merge : exemple sur deux branches distinctes.

```
$ ls
dir fichier2.txt foo.txt

$ git switch maBranche
$ ls
dir fichier1.txt foo.txt

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"
```



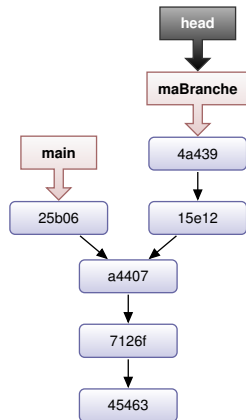
Merge : exemple sur deux branches distinctes.

```
$ ls
dir fichier2.txt foo.txt

$ git switch maBranche
$ ls
dir fichier1.txt foo.txt

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main
```



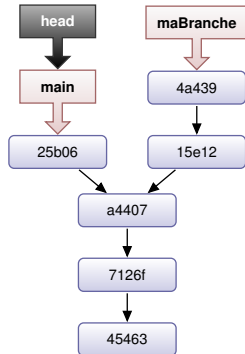
Merge : exemple sur deux branches distinctes.

```
$ ls
dir fichier2.txt foo.txt

$ git switch maBranche
$ ls
dir fichier1.txt foo.txt

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main
```



Merge : exemple sur deux branches distinctes.

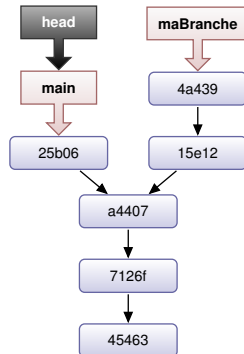
```
$ ls
dir fichier2.txt foo.txt

$ git switch maBranche
$ ls
dir fichier1.txt foo.txt

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main

$ git merge maBranche
$ ls
dir fichier1.txt fichier2.txt foo.txt
```



Merge : exemple sur deux branches distinctes.

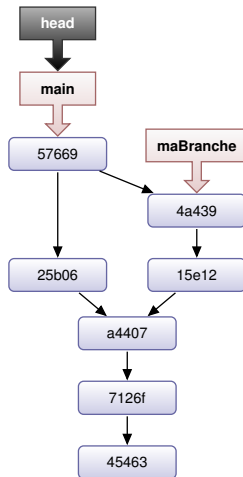
```
$ ls
dir fichier2.txt foo.txt

$ git switch maBranche
$ ls
dir fichier1.txt foo.txt

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main

$ git merge maBranche
$ ls
dir fichier1.txt fichier2.txt foo.txt
```



Merge : exemple sur deux branches distinctes.

```
$ ls
dir fichier2.txt foo.txt

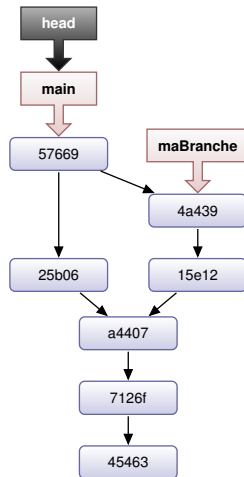
$ git switch maBranche
$ ls
dir fichier1.txt foo.txt

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main

$ git merge maBranche
$ ls
dir fichier1.txt fichier2.txt foo.txt

$ git branch -d maBranche
```



Merge : exemple sur deux branches distinctes.

```
$ ls
dir fichier2.txt foo.txt

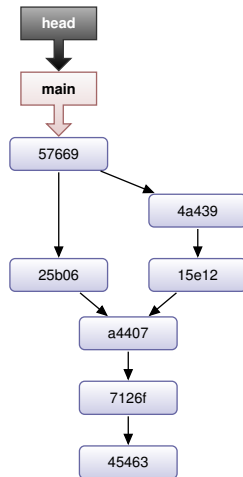
$ git switch maBranche
$ ls
dir fichier1.txt foo.txt

$ echo "titi" > fichier1.txt
$ git commit -am "New fichier1.txt"

$ git switch main

$ git merge maBranche
$ ls
dir fichier1.txt fichier2.txt foo.txt

$ git branch -d maBranche
```



Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

Les commits

Les branches

Les merges

Les rebases

Les remords

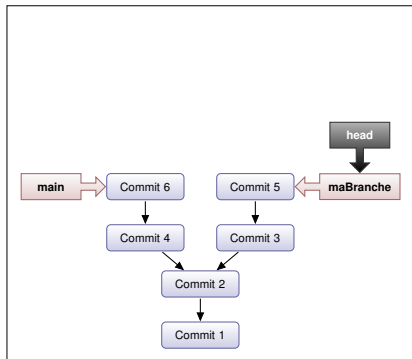
Utilisation de l'historique

Synchronisation avec les dépôts distants.

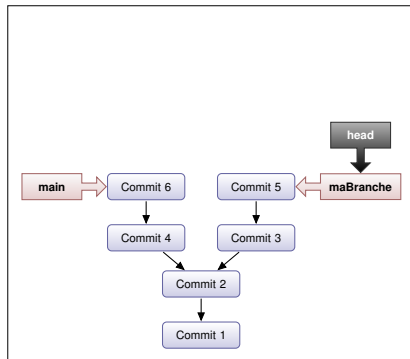
Les outils graphiques

Conclusion

Rebase vs Merge.

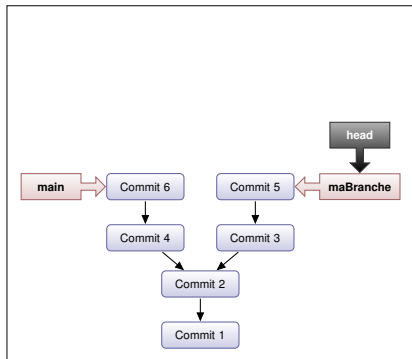


```
$ git switch maBranche  
$ git merge main
```

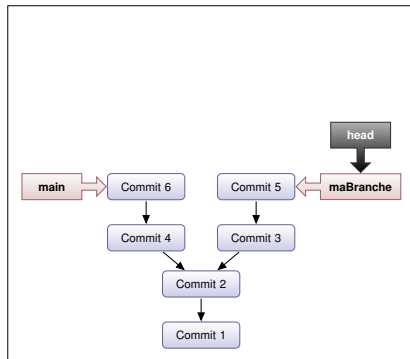


```
$ git switch maBranche  
$ git rebase main
```

Rebase vs Merge.

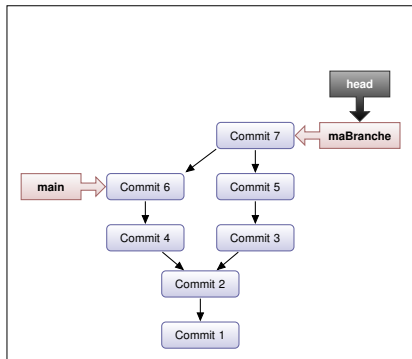


```
$ git switch maBranche  
$ git merge main
```

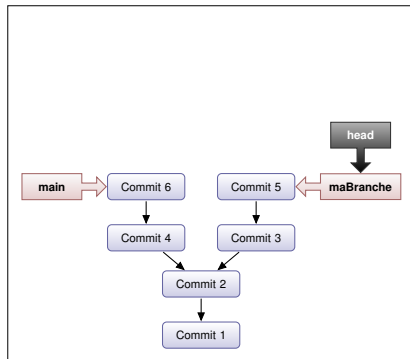


```
$ git switch maBranche  
$ git rebase main
```

Rebase vs Merge.

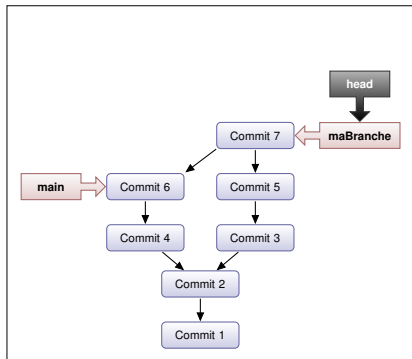


```
$ git switch maBranche  
$ git merge main
```

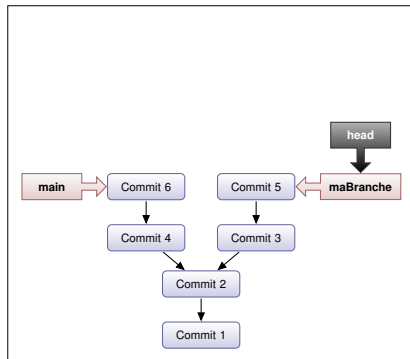


```
$ git switch maBranche  
$ git rebase main
```

Rebase vs Merge.

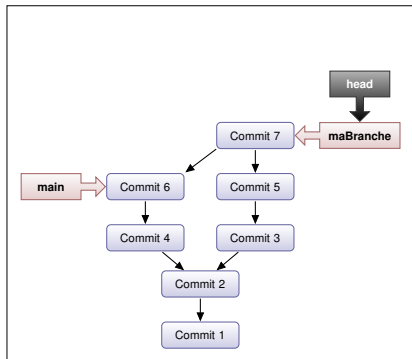


```
$ git switch maBranche  
$ git merge main
```

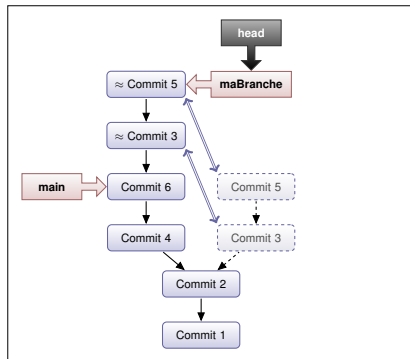


```
$ git switch maBranche  
$ git rebase main
```

Rebase vs Merge.

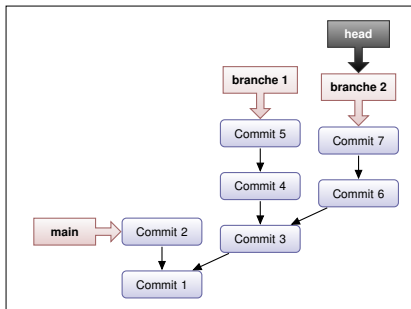


```
$ git switch maBranche  
$ git merge main
```



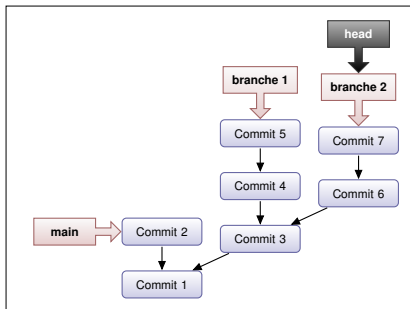
```
$ git switch maBranche  
$ git rebase main
```

Rebase vs Merge.



```
$ git switch branche2
```

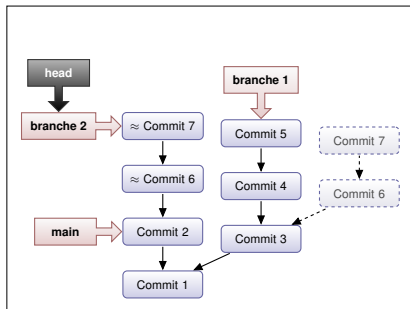
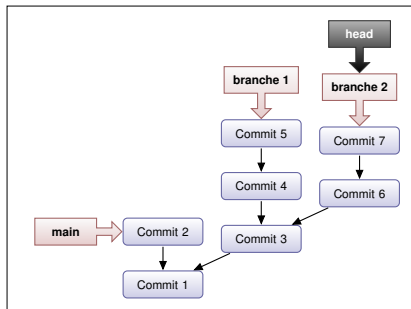

Rebase vs Merge.



```
$ git switch branche2
```

```
$ git rebase --onto main branche1 branche2
```

Rebase vs Merge.



```
$ git switch branche2
```

```
$ git rebase --onto main branche1 branche2
```

Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

- Les commits

- Les branches

- Les merges

- Les rebases

- Les remords

- Utilisation de l'historique

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

En cas d'erreur sur un commit, git propose 3 types de correction :

revert : pour annuler un commit par un autre commit.

amend : modifier le dernier commit.

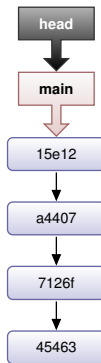
reset : pour rétablir la situation d'un ancien commit.

Attention

Si l'erreur a déjà été rendue publique, la seule bonne pratique est le **revert**. Les autres solutions peuvent conduire à des incohérences.

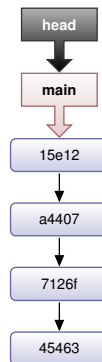
Amend : modification du dernier commit.

```
$ ls  
foo.txt dir
```



Amend : modification du dernier commit.

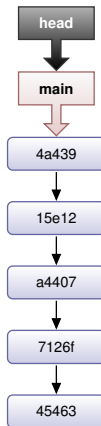
```
$ ls  
foo.txt dir  
  
$ touch bar.txt  
$ git commit -m "Ajou d'un fichier."
```



Amend : modification du dernier commit.

```
$ ls
foo.txt dir

$ touch bar.txt
$ git commit -m "Ajou d'un fichier."
```

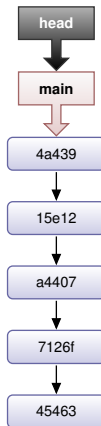


Amend : modification du dernier commit.

```
$ ls
foo.txt dir

$ touch bar.txt
$ git commit -m "Ajou d'un fichier."

$ git add bar.txt
$ git commit --amend -m "Ajout d'un fichier."
```

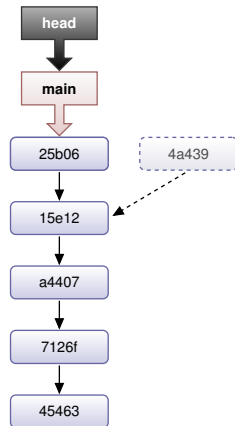


Amend : modification du dernier commit.

```
$ ls
foo.txt dir

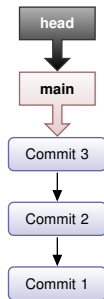
$ touch bar.txt
$ git commit -m "Ajou d'un fichier."

$ git add bar.txt
$ git commit --amend -m "Ajout d'un fichier."
```



Git revert : annulation par commit.

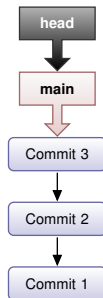
```
$ git branch main
$ cat fichier1.txt
Premiere version de F1
$ cat fichier2.txt
Premiere version de F2
```



Git revert : annulation par commit.

```
$ git branch main
$ cat fichier1.txt
Premiere version de F1
$ cat fichier2.txt
Premiere version de F2

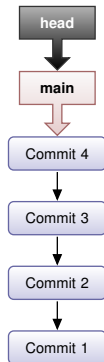
$ echo "Deuxieme version de F1" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"
```



Git revert : annulation par commit.

```
$ git branch main
$ cat fichier1.txt
Premiere version de F1
$ cat fichier2.txt
Premiere version de F2

$ echo "Deuxieme version de F1" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"
```

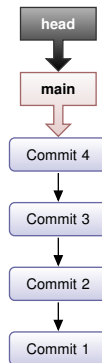


Git revert : annulation par commit.

```
$ git branch main
$ cat fichier1.txt
Premiere version de F1
$ cat fichier2.txt
Premiere version de F2

$ echo "Deuxieme version de F1" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "Deuxieme version de F2" > fichier2.txt
$ git add fichier2.txt
$ git commit -m "Add fichier2.txt"
```

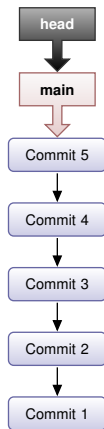


Git revert : annulation par commit.

```
$ git branch main
$ cat fichier1.txt
Premiere version de F1
$ cat fichier2.txt
Premiere version de F2

$ echo "Deuxieme version de F1" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "Deuxieme version de F2" > fichier2.txt
$ git add fichier2.txt
$ git commit -m "Add fichier2.txt"
```



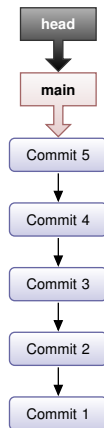
Git revert : annulation par commit.

```
$ git branch main
$ cat fichier1.txt
Premiere version de F1
$ cat fichier2.txt
Premiere version de F2

$ echo "Deuxieme version de F1" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "Deuxieme version de F2" > fichier2.txt
$ git add fichier2.txt
$ git commit -m "Add fichier2.txt"

$ git revert HEAD^
$ cat fichier1.txt
Premiere version de F1
$ cat fichier2.txt
Deuxieme version de F2
```



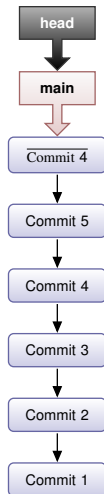
Git revert : annulation par commit.

```
$ git branch main
$ cat fichier1.txt
Premiere version de F1
$ cat fichier2.txt
Premiere version de F2

$ echo "Deuxieme version de F1" > fichier1.txt
$ git add fichier1.txt
$ git commit -m "Add fichier1.txt"

$ echo "Deuxieme version de F2" > fichier2.txt
$ git add fichier2.txt
$ git commit -m "Add fichier2.txt"

$ git revert HEAD^
$ cat fichier1.txt
Premiere version de F1
$ cat fichier2.txt
Deuxieme version de F2
```



Git reset : "suppression" de commit.

git reset permet de "supprimer" un ou plusieurs commits.

Dans les faits, les objets liés à ces commits ne seront vraiment effacés qu'après un appel à **git gc** et s'ils sont suffisamment vieux (15 jours par défaut).

Il existe trois types de reset :

1. **git reset --hard** :

- ▶ restore la référence du commit (de la branche active)
- ▶ restore l'index
- ▶ restore les données

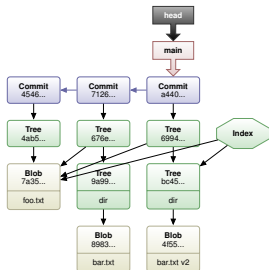
2. **git reset** :

- ▶ restore la référence du commit (de la branche active)
- ▶ restore l'index

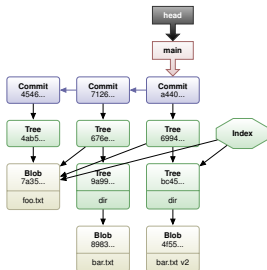
3. **git reset --soft** :

- ▶ restore la référence du commit (de la branche active)

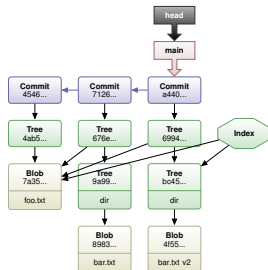
Les différents type de reset.



```
$ ls -R
.: foo.txt
dir: bar.txt
```



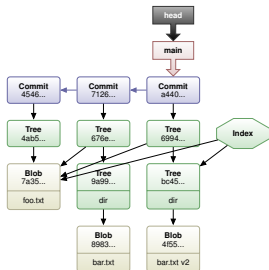
```
$ ls -R
.: foo.txt
dir: bar.txt
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

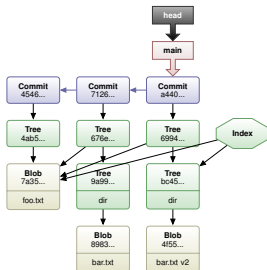
```
$ ls -R
.: foo.txt
dir: bar.txt
```

Les différents type de reset.

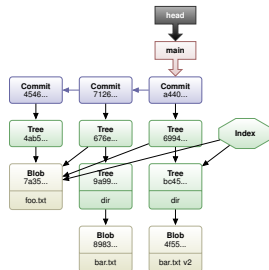


```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ git reset --hard
4546
```



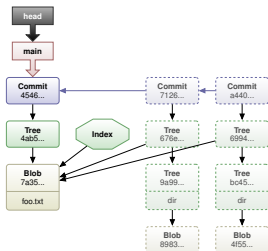
```
$ ls -R
.: foo.txt
dir: bar.txt
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

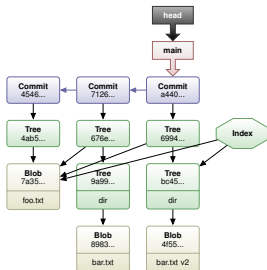
```
$ ls -R
.: foo.txt
dir: bar.txt
```

Les différents type de reset.

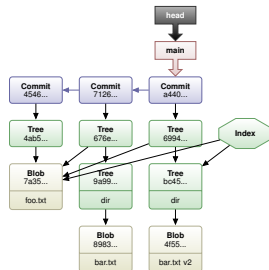


```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ git reset --hard
4546
```



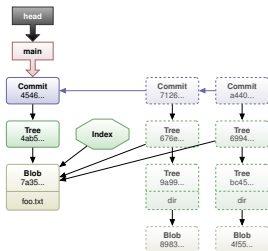
```
$ ls -R
.: foo.txt
dir: bar.txt
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ ls -R
.: foo.txt
dir: bar.txt
```

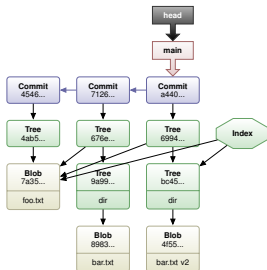
Les différents type de reset.



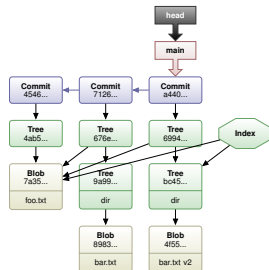
```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ git reset --hard
4546
```

```
$ ls -R
.: foo.txt
```



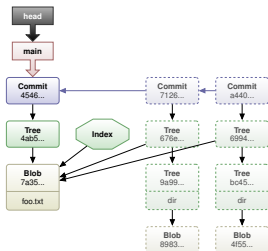
```
$ ls -R
.: foo.txt
dir: bar.txt
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ ls -R
.: foo.txt
dir: bar.txt
```

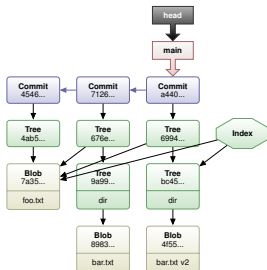
Les différents type de reset.



```
$ ls -R
.: foo.txt
dir: bar.txt
```

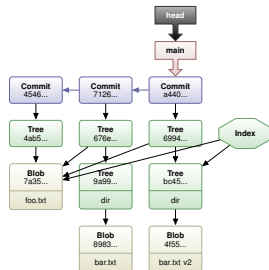
```
$ git reset --hard
4546
```

```
$ ls -R
.: foo.txt
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

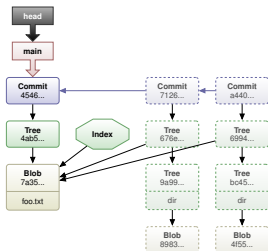
```
$ git reset 4546
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ ls -R
.: foo.txt
dir: bar.txt
```

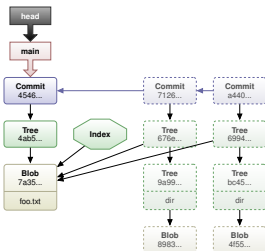
Les différents type de reset.



```
$ ls -R
.: foo.txt
dir: bar.txt
```

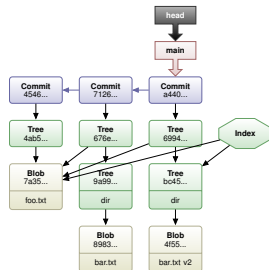
```
$ git reset --hard
4546
```

```
$ ls -R
.: foo.txt
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

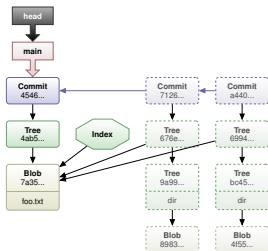
```
$ git reset 4546
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ ls -R
.: foo.txt
dir: bar.txt
```

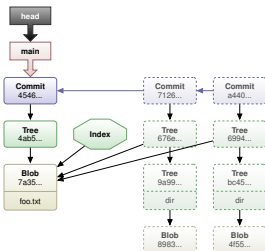
Les différents type de reset.



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ git reset --hard
4546
```

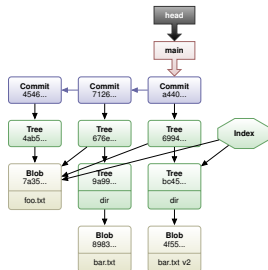
```
$ ls -R
.: foo.txt
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ git reset 4546
```

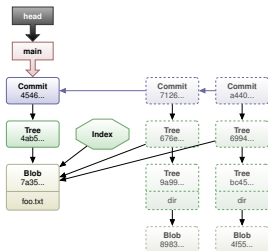
```
$ ls -R
.: foo.txt
dir: bar.txt
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ ls -R
.: foo.txt
dir: bar.txt
```

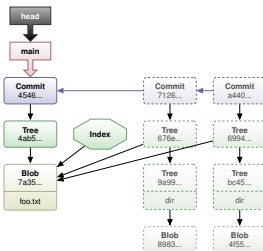

Les différents type de reset.



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ git reset --hard
4546
```

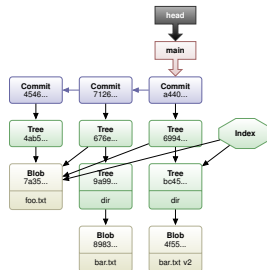
```
$ ls -R
.: foo.txt
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ git reset 4546
```

```
$ ls -R
.: foo.txt
dir: bar.txt
```

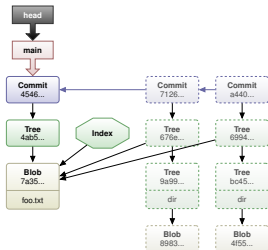


```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ git reset --soft
4546
```

```
$ ls -R
.: foo.txt
dir: bar.txt
```

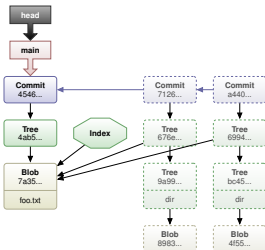
Les différents type de reset.



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ git reset --hard
4546
```

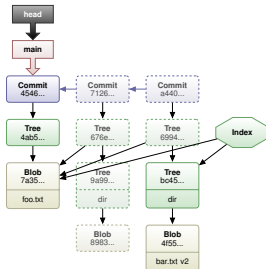
```
$ ls -R
.: foo.txt
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ git reset 4546
```

```
$ ls -R
.: foo.txt
dir: bar.txt
```



```
$ ls -R
.: foo.txt
dir: bar.txt
```

```
$ git reset --soft
4546
```

```
$ ls -R
.: foo.txt
dir: bar.txt
```

Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

- Les commits

- Les branches

- Les merges

- Les rebases

- Les remords

- Utilisation de l'historique

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

Comparaison : git diff

- ▶ Différences entre le répertoire de travail et l'index :

```
$ git diff
```

- ▶ Différences entre HEAD et l'index :

```
$ git diff --staged
```

- ▶ Différences entre répertoire de travail et HEAD :

```
$ git diff HEAD
```

- ▶ Différences entre répertoire de travail et un autre commit :

```
$ git diff <commit_1>
```

- ▶ Différences entre deux commit :

Information sur un commit : git show

```
$ git show
commit 7a618a3f3c23262ad281c9afe69e145ef867d43b
Author: Julien SOPENA <julien.sopena@lip6.fr>
Date: Mon Oct 25 03:55:27 2010 +0200

Ceci est un petit exemple de commit.

diff -git a/test b/test
index 808a2c4..99810fa 100644
-- a/test
+++ b/test
@@ -1,3 +1,3 @@
Ligne de texte non modifié par ce commit.
-Ligne de texte supprimée.
+Nouvelle ligne de texte
Suite du texte non modifié.
```

Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Principe des DVCS : Gestionnaire de versions décentralisé.

Les dépôts distants.

Modèles de travail coopératif

Les outils graphiques

Conclusion

Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Principe des DVCS : Gestionnaire de versions décentralisé.

Les dépôts distants.

Modèles de travail coopératif

Les outils graphiques

Conclusion

Dépôts centralisés

CVS ou Subversion



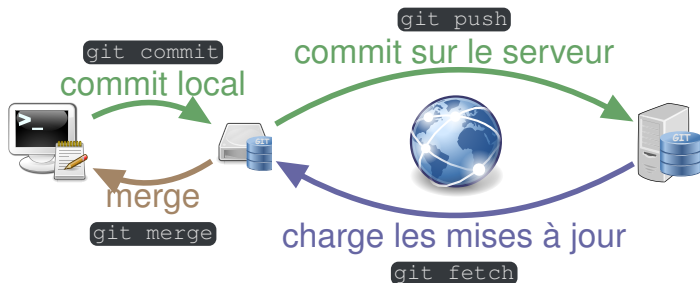
Dépôts centralisés

CVS ou Subversion

- ▶ Toutes les versions des fichiers sont entreposées dans un unique dépôt accessible aux clients autorisés
- ▶ Les clients ne travaillent que sur une partie des données, en général, une simple branche
- ▶ Chaque changement de branche nécessite un téléchargement de l'ensemble des données de la branche.
- ▶ Impossibilité de versionner *Off-line*
- ▶ Inclure un contributeur nécessite d'ouvrir le dépôt en écriture.

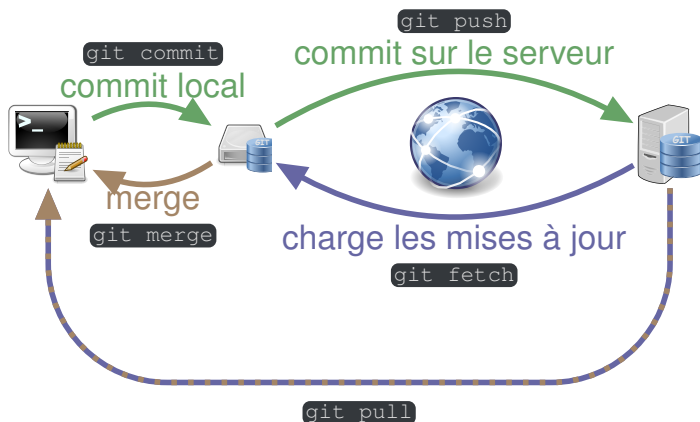
Dépôts décentralisés

Arch, Bazar-Ng, Git, monotone etc.



Dépôts décentralisés

Arch, Bazar-Ng, Git, monotone etc.



Dépôts décentralisés

Arch, Bazar-Ng, Git, monotone etc.

- ▶ Chaque client a l'ensemble des fichiers dans son dépôt local :
 - ▶ On peut travailler *Off-Line* (à la plage, à la montagne,...)
 - ▶ Le changement de branche est rapide et est donc, une des méthodes de développement : **utiliser les branches**
- ▶ Les seules actions du client nécessitant un accès au dépôt distant sont :
 - ▶ la mise à jour du dépôt local depuis l'extérieur
 - ▶ l'envoi d'information.
- ▶ **Le client peut versionner en local !**

Definition

On appelle **conflits**, toute modification d'un fichier dans un dépôt qui n'a pas été élaborée à partir de la version actuelle du fichier sur ce dépôt, *i.e.*, lorsqu'il y a une modification de ce fichier sur le dépôt pendant son édition.

Apparition sur CVS/Subversion :

- ▶ au téléchargement des modifications : cvs **update** ;
- ▶ au commit avec un refus d'enregistrer sur le dépôt.

Apparition sur Git et les autres :

- ▶ Uniquement lors des **fusions** de branches :
locale/locale, locale/distante ou distante/locale.

Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Principe des DVCS : Gestionnaire de versions décentralisé.

Les dépôts distants.

Modèles de travail coopératif

Les outils graphiques

Conclusion

Un projet décentralisé possède deux types de branches :

Definition

On appelle **branche distante**, une branche qui pointe sur des dépôts distants en lecture et/ou écriture. Ces dépôts distants peuvent être référencés par une ou plusieurs personnes.

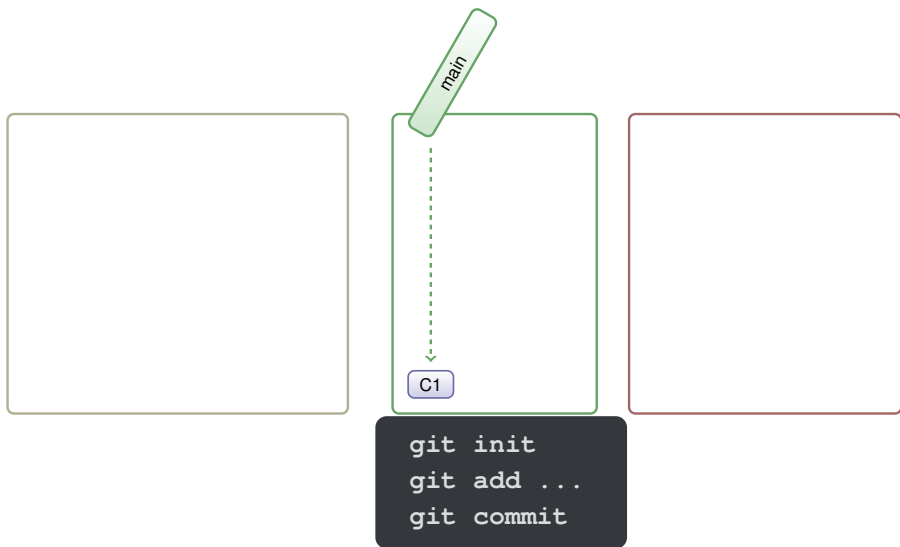
Definition

On appelle **branche locale**, une branche propre au dépôt local. Pour être envoyées, les données d'une telle branche doivent être fusionnées avec une branche distante.

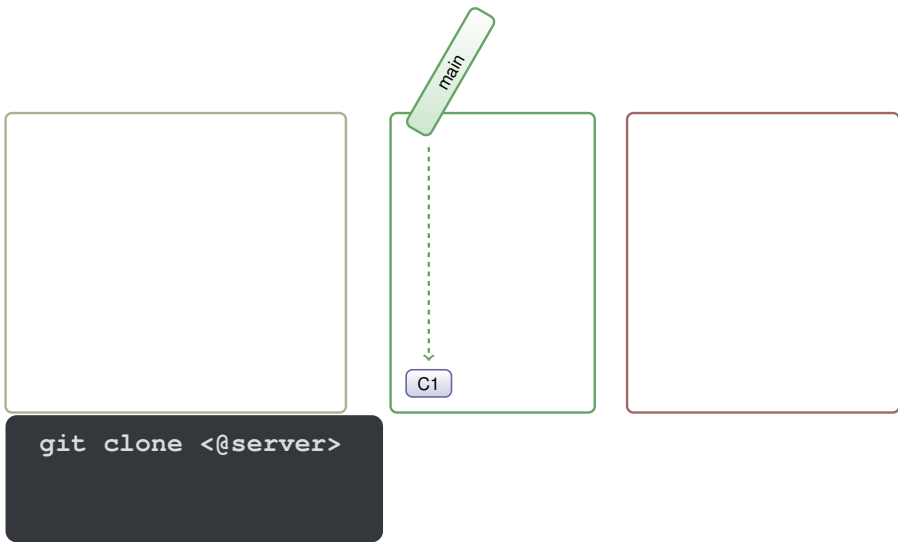
Dépôts distants et gestion de la concurrence.



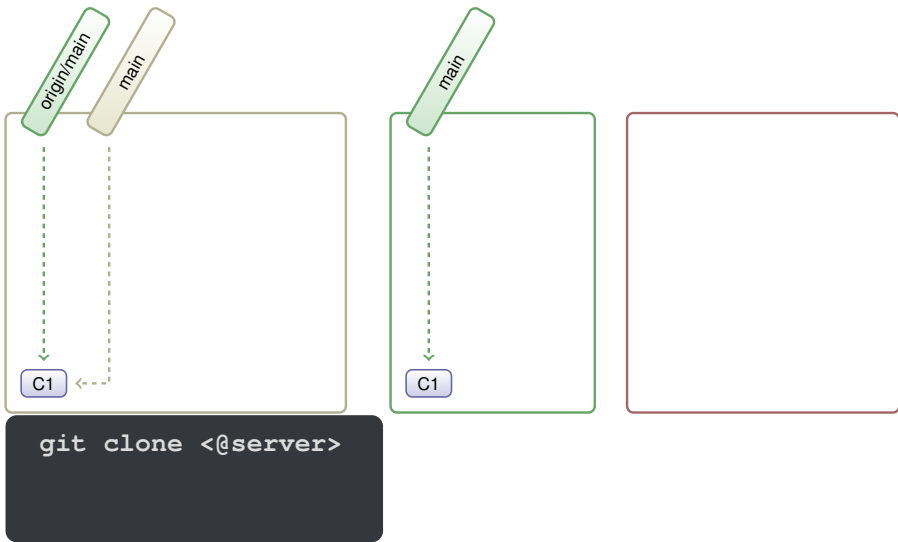
Dépôts distants et gestion de la concurrence.



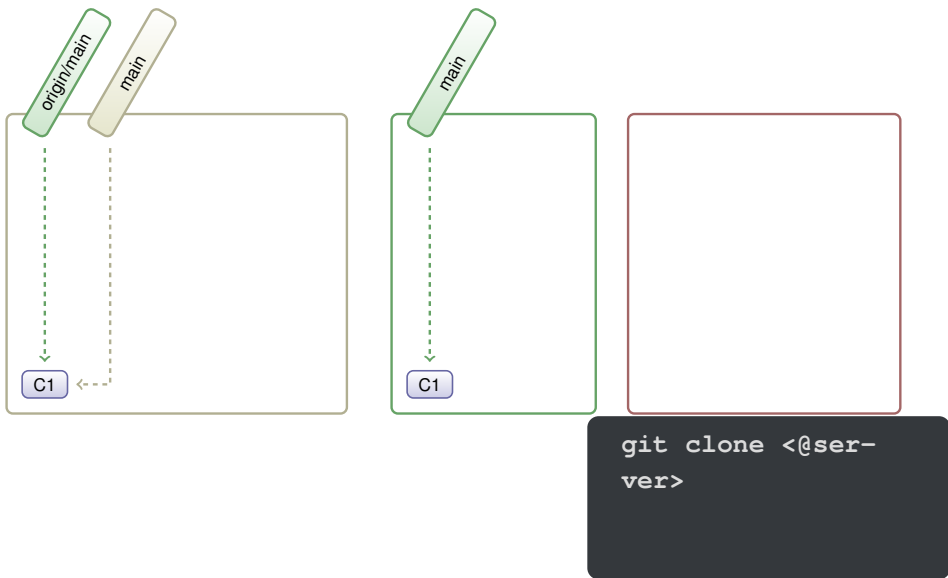
Dépôts distants et gestion de la concurrence.



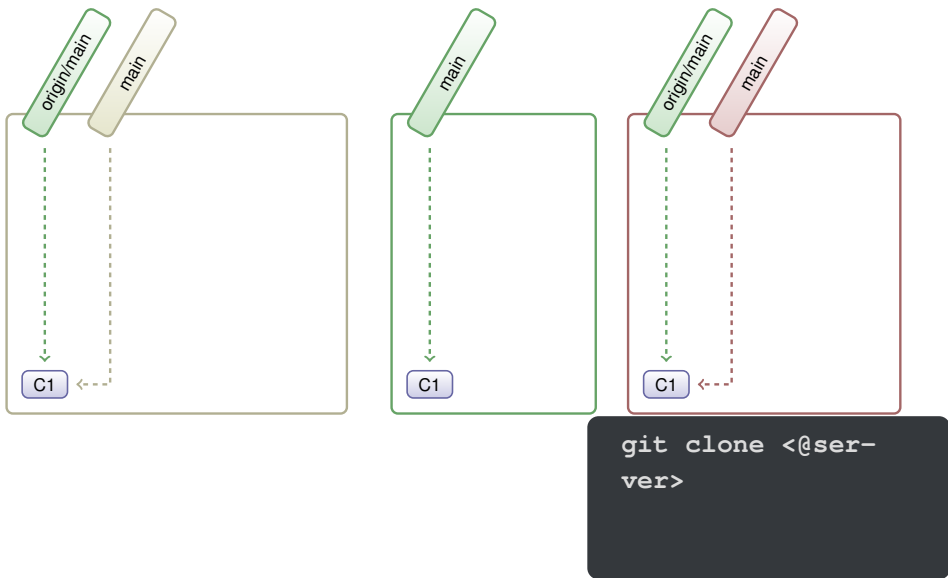
Dépôts distants et gestion de la concurrence.



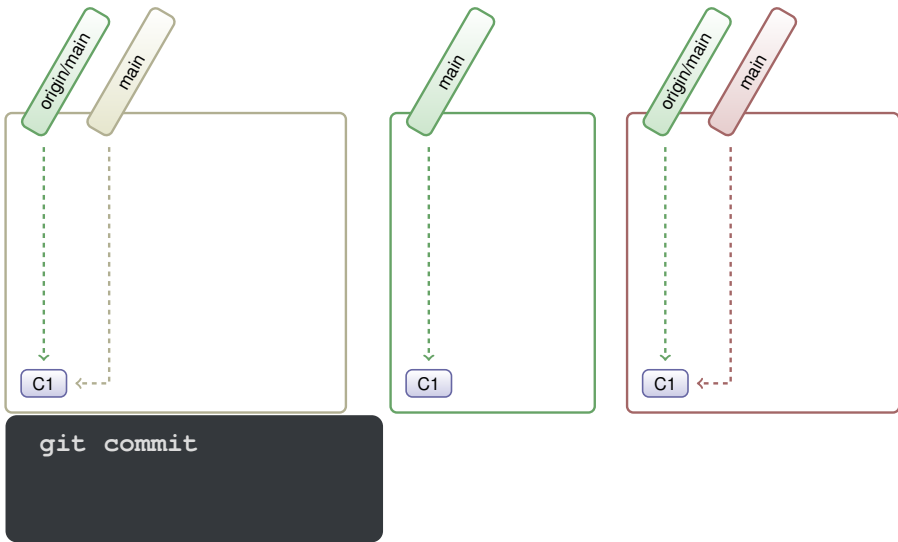
Dépôts distants et gestion de la concurrence.



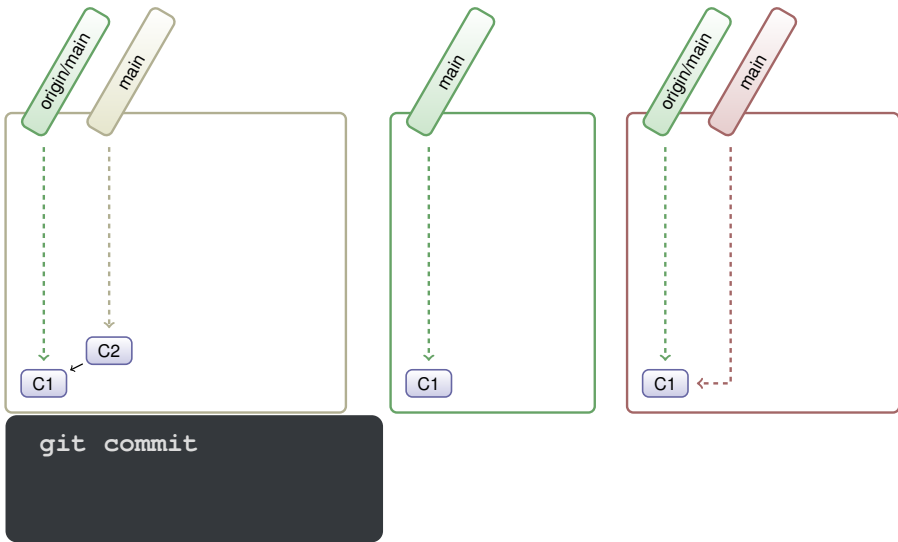
Dépôts distants et gestion de la concurrence.



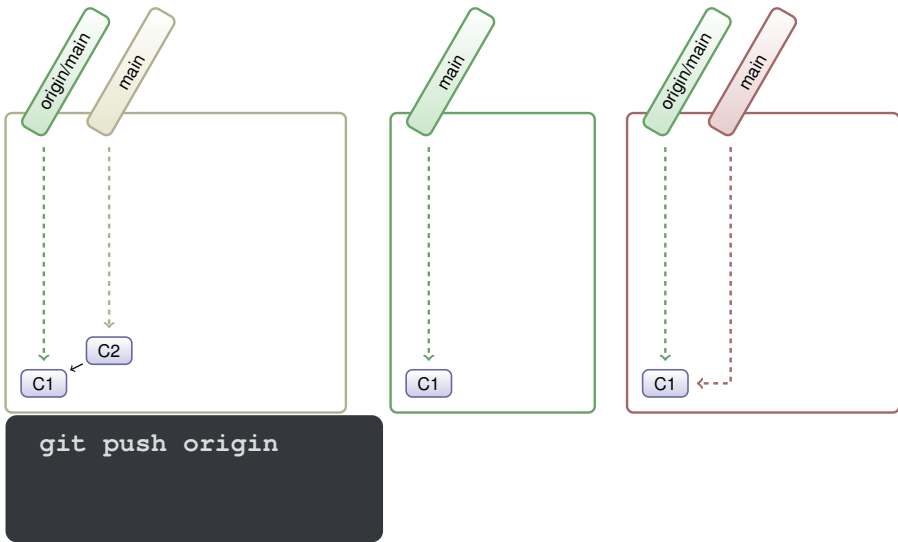
Dépôts distants et gestion de la concurrence.



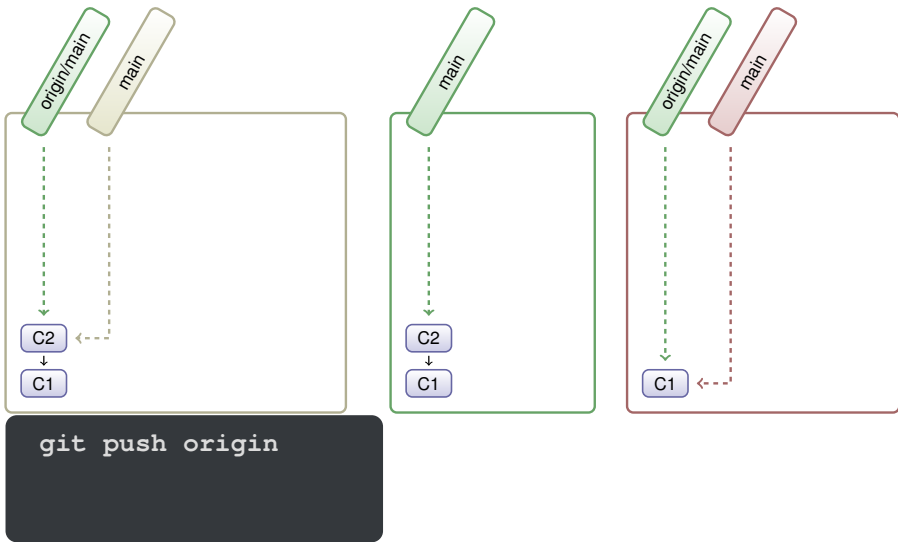
Dépôts distants et gestion de la concurrence.



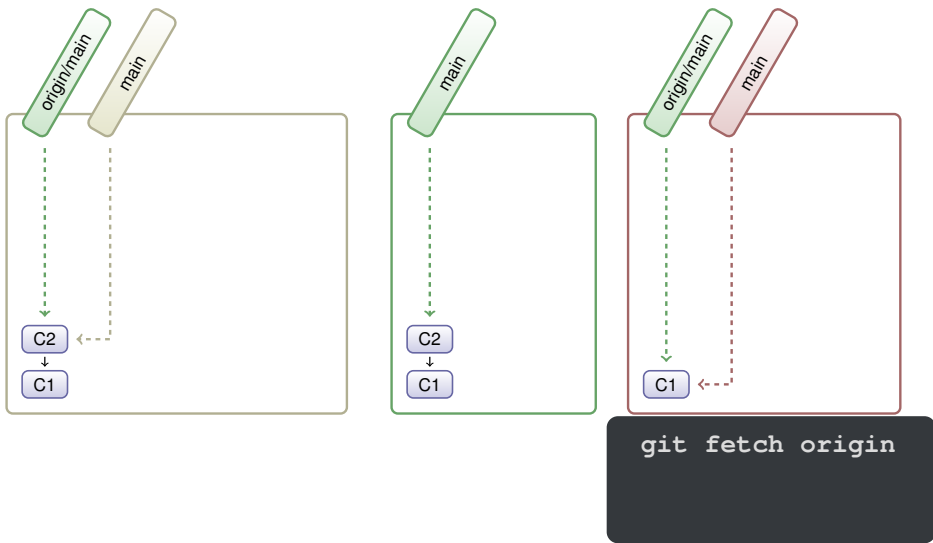
Dépôts distants et gestion de la concurrence.



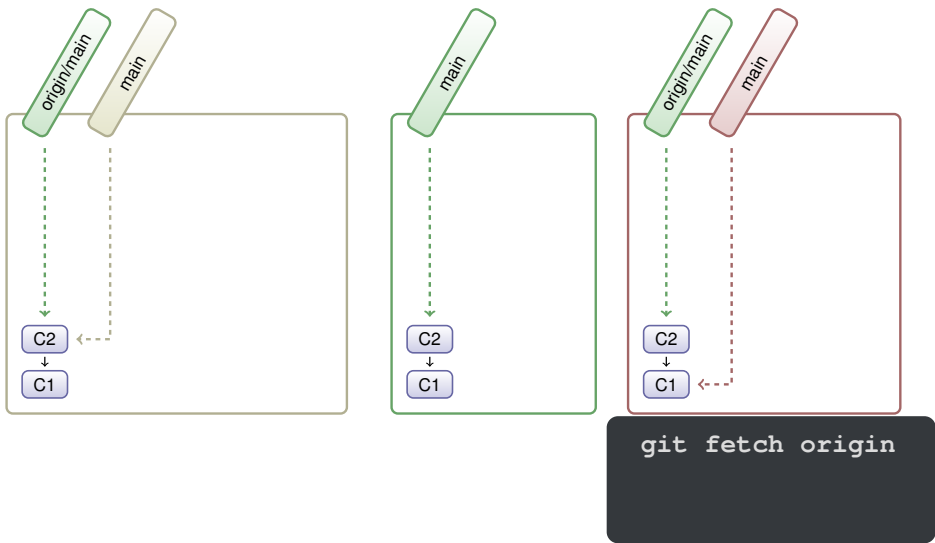
Dépôts distants et gestion de la concurrence.



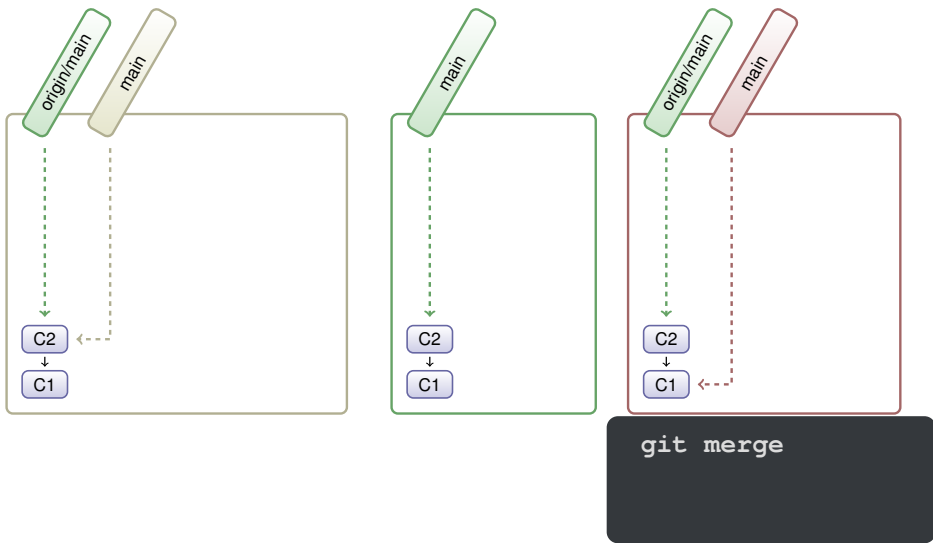
Dépôts distants et gestion de la concurrence.



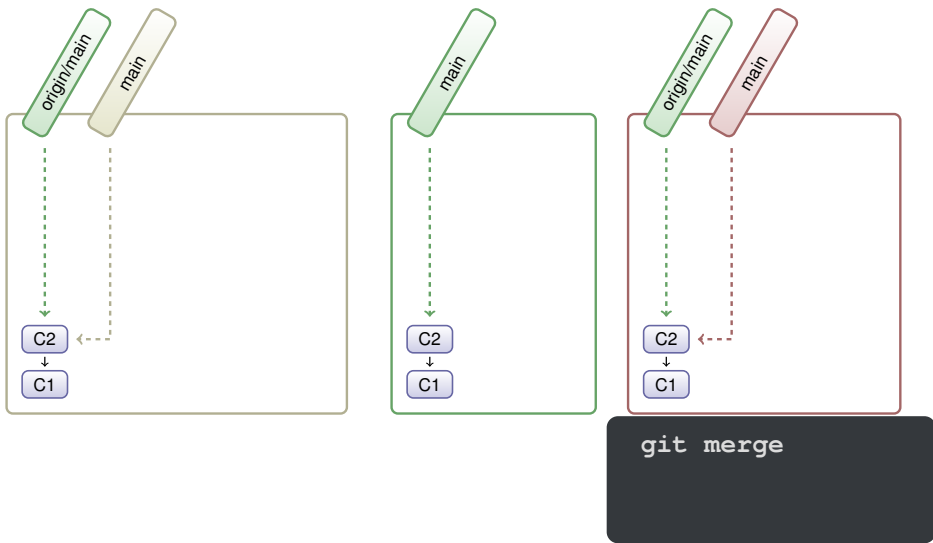
Dépôts distants et gestion de la concurrence.



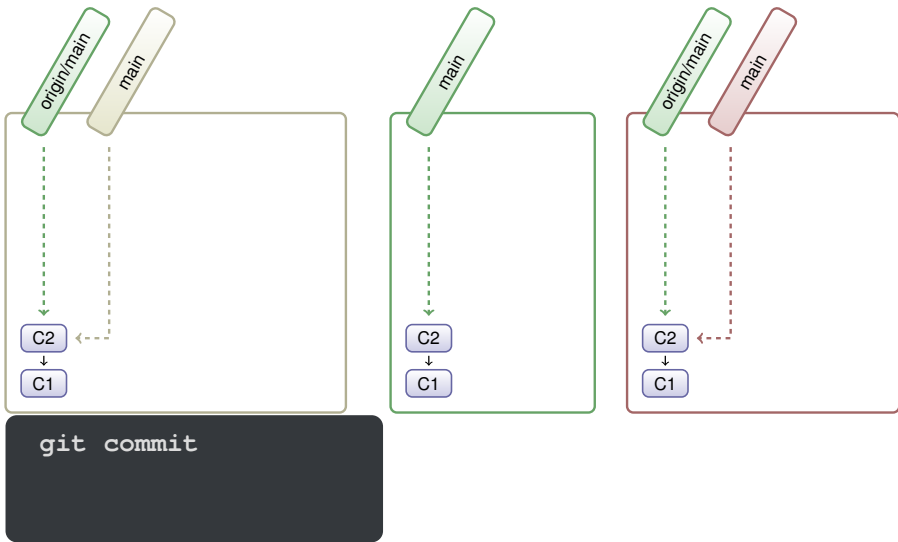
Dépôts distants et gestion de la concurrence.



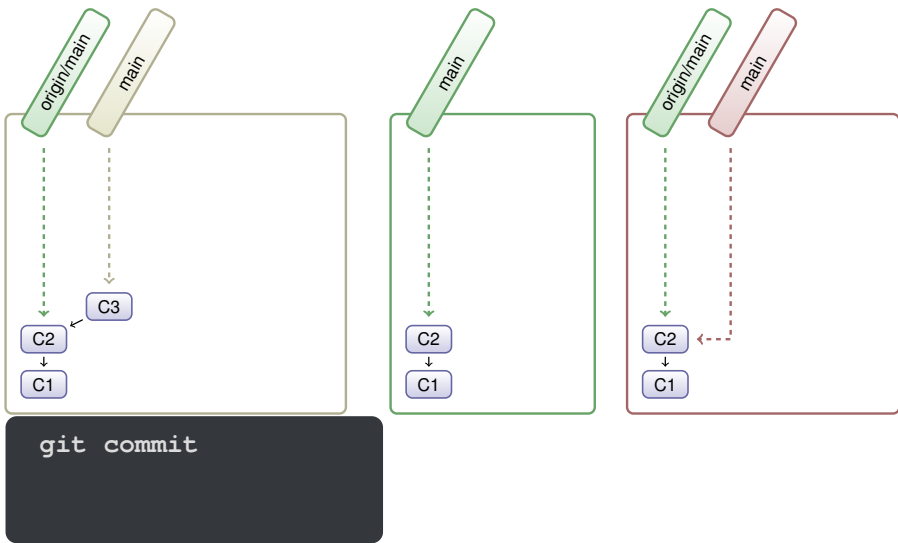
Dépôts distants et gestion de la concurrence.



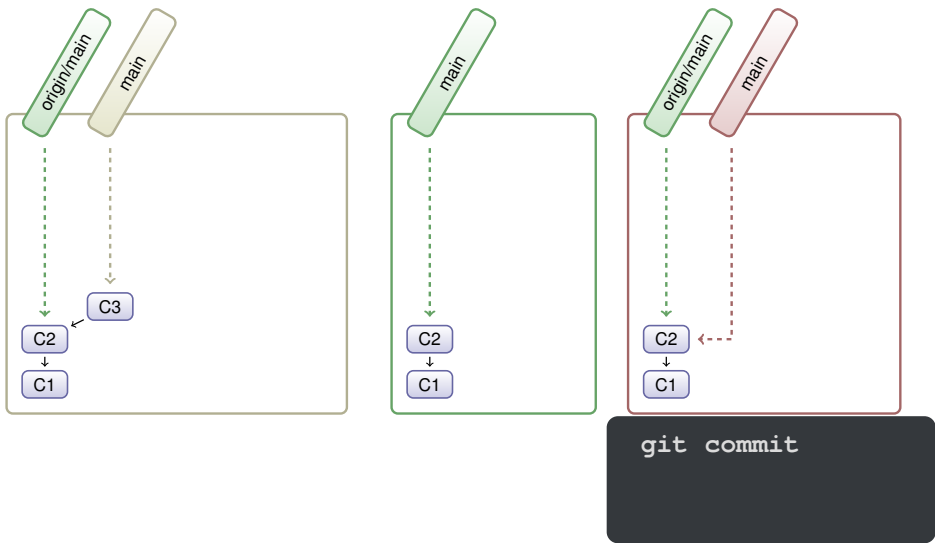
Dépôts distants et gestion de la concurrence.



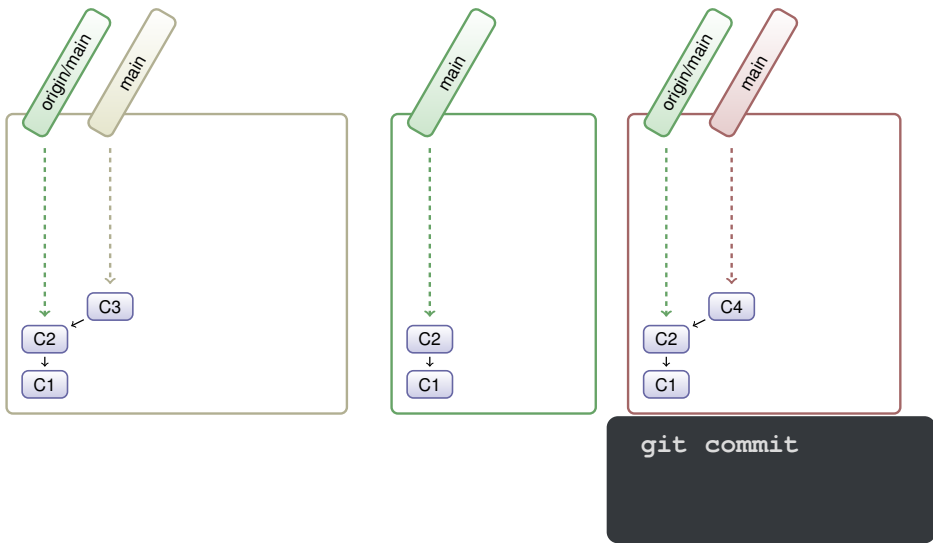
Dépôts distants et gestion de la concurrence.



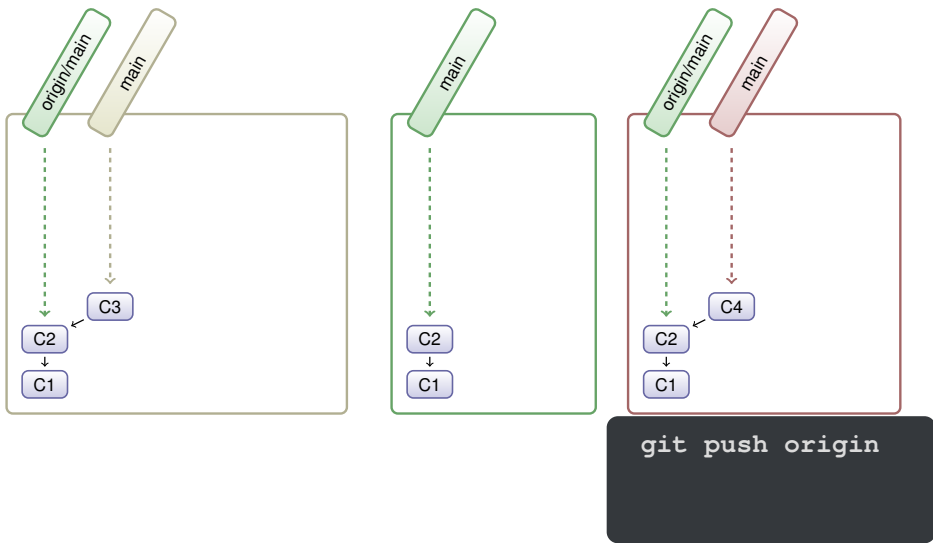
Dépôts distants et gestion de la concurrence.



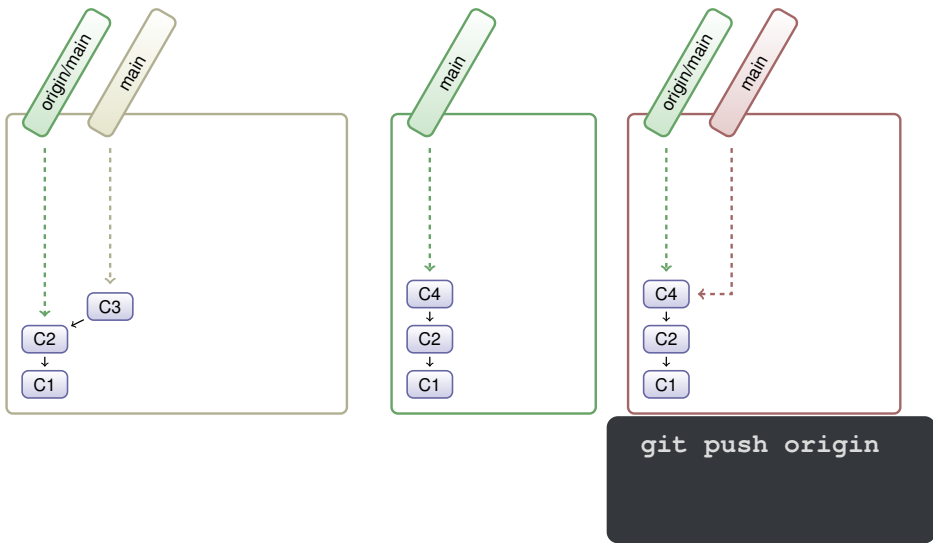
Dépôts distants et gestion de la concurrence.



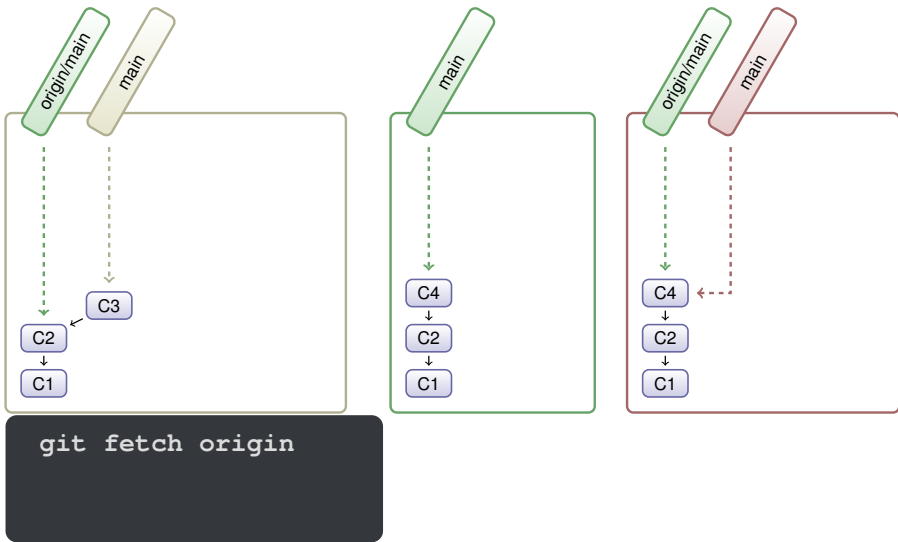
Dépôts distants et gestion de la concurrence.



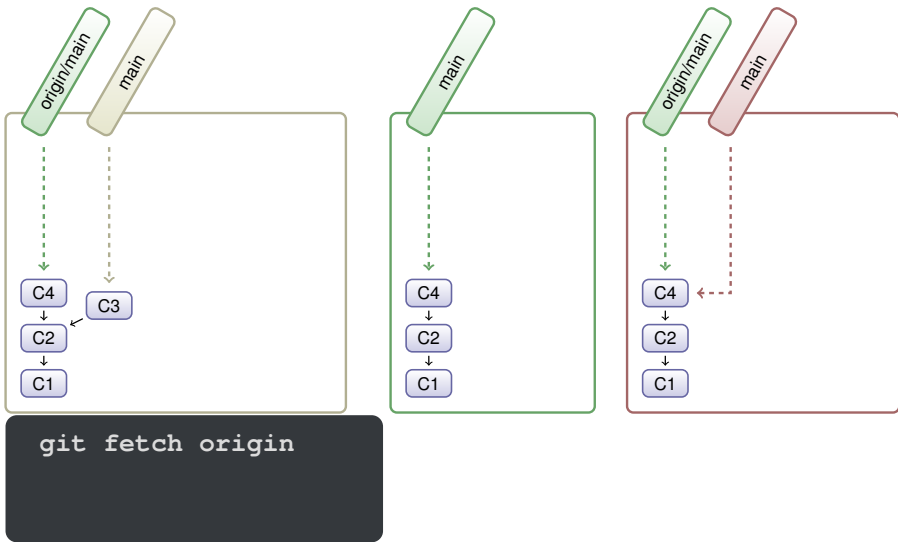
Dépôts distants et gestion de la concurrence.



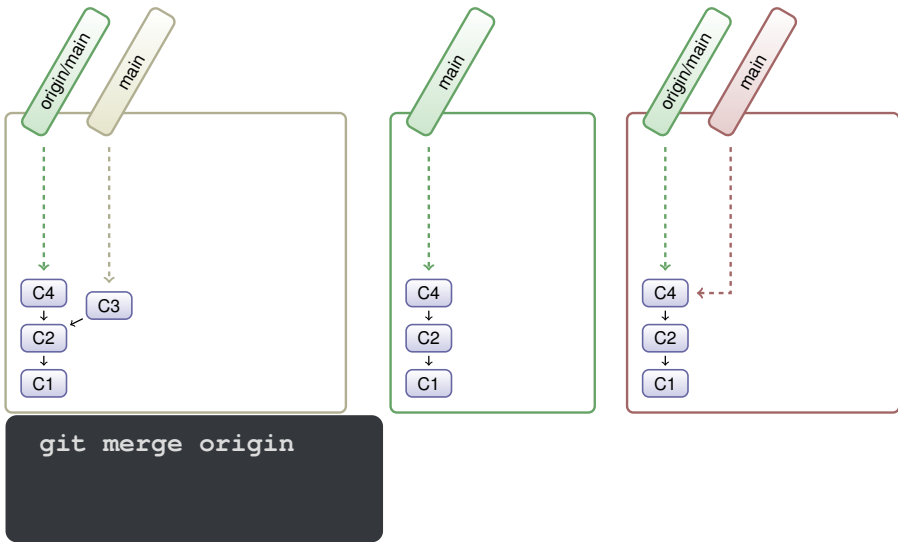
Dépôts distants et gestion de la concurrence.



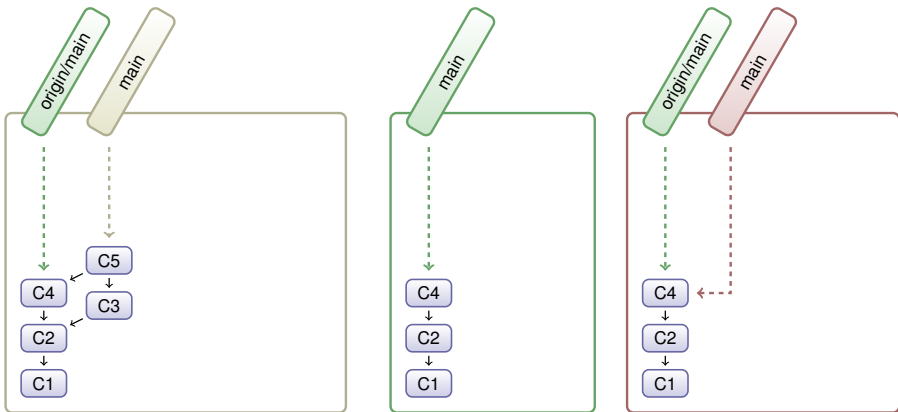
Dépôts distants et gestion de la concurrence.



Dépôts distants et gestion de la concurrence.

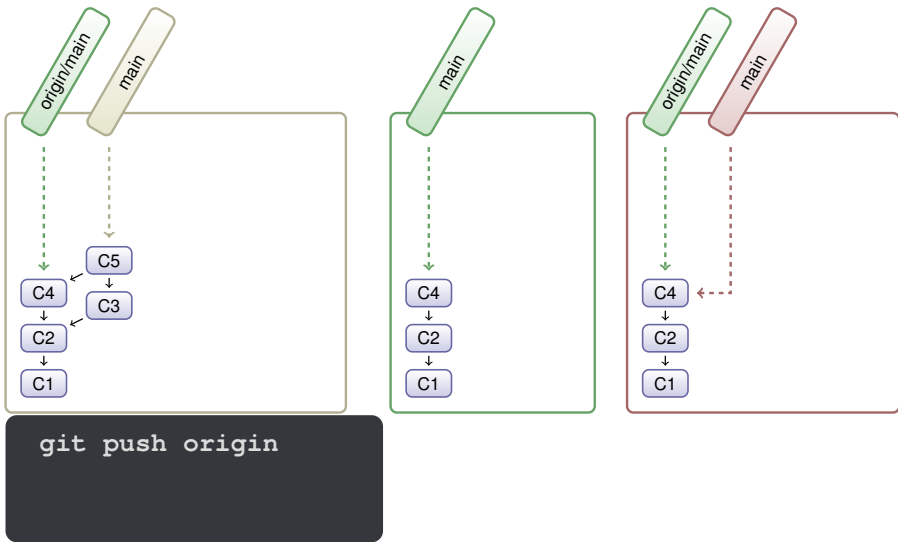


Dépôts distants et gestion de la concurrence.

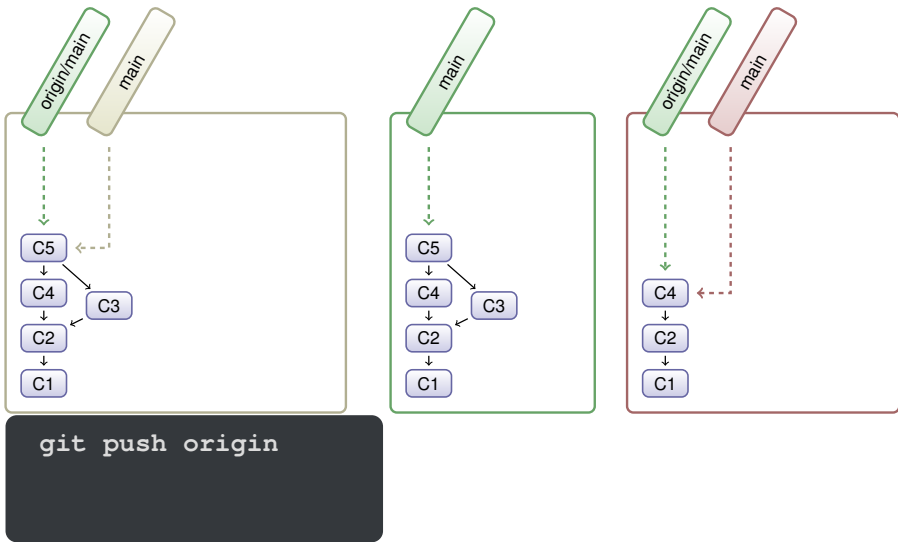


```
git merge origin
```

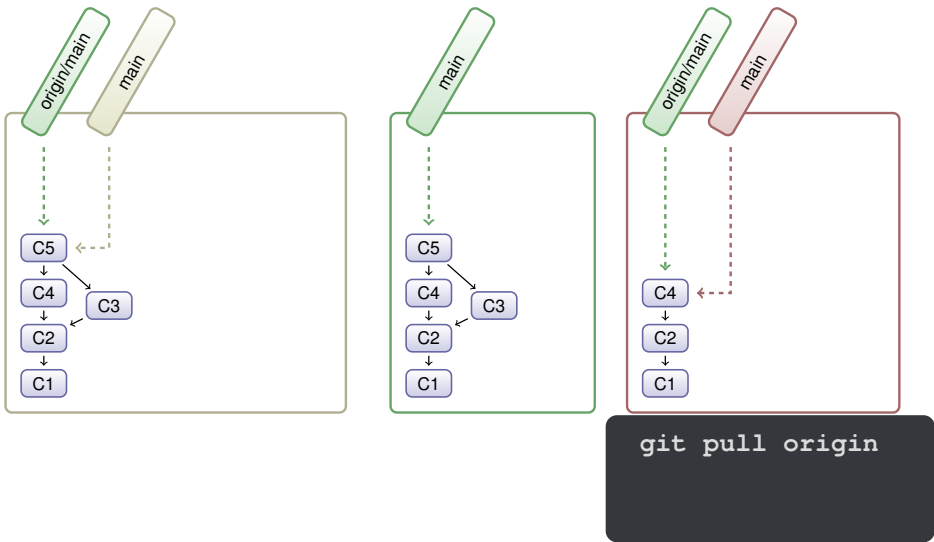
Dépôts distants et gestion de la concurrence.



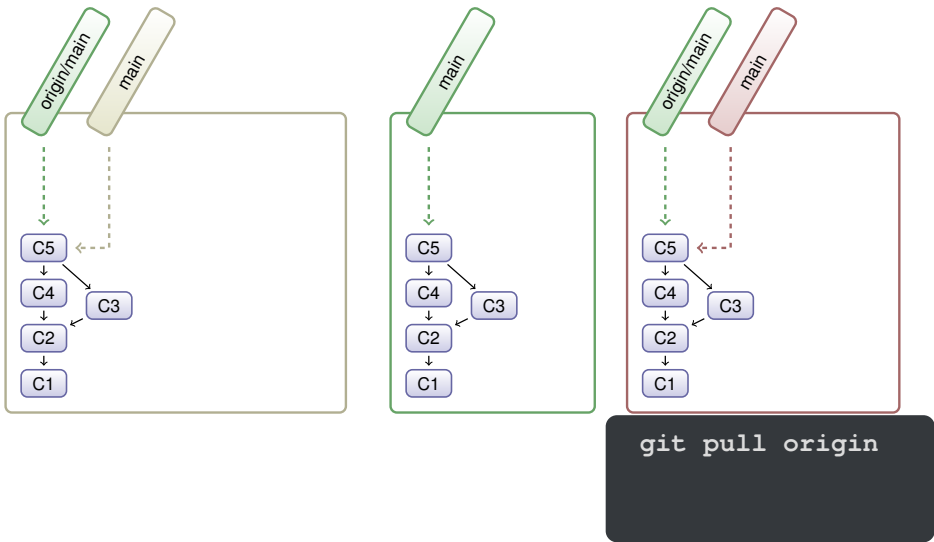
Dépôts distants et gestion de la concurrence.



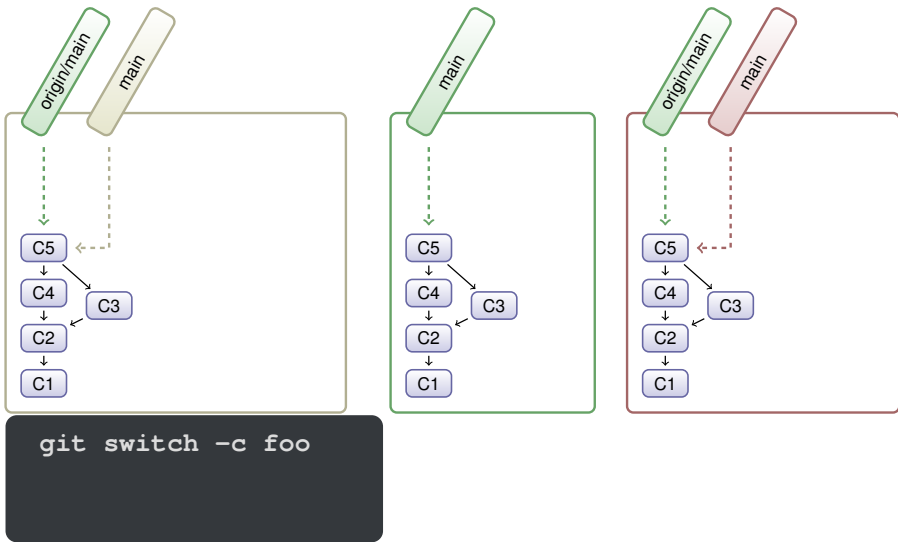
Dépôts distants et gestion de la concurrence.



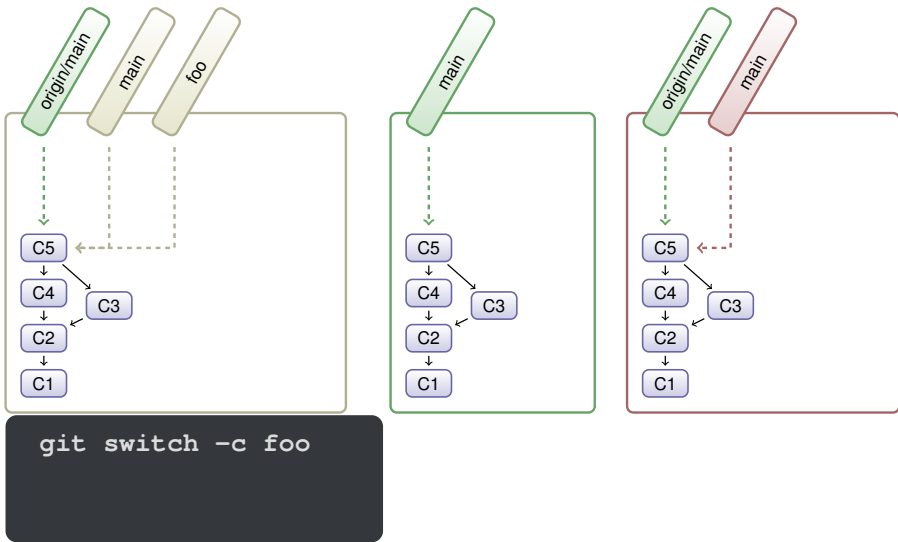
Dépôts distants et gestion de la concurrence.



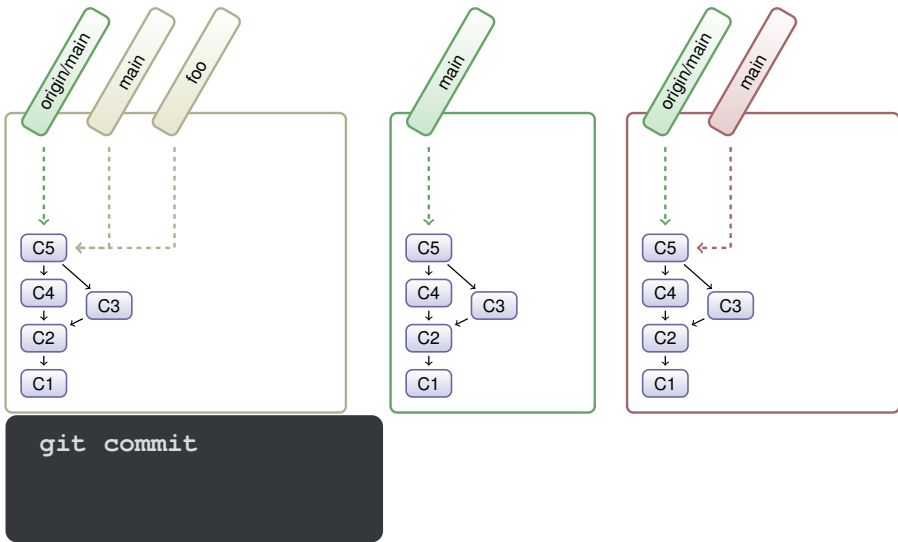
Dépôts distants et gestion de la concurrence.



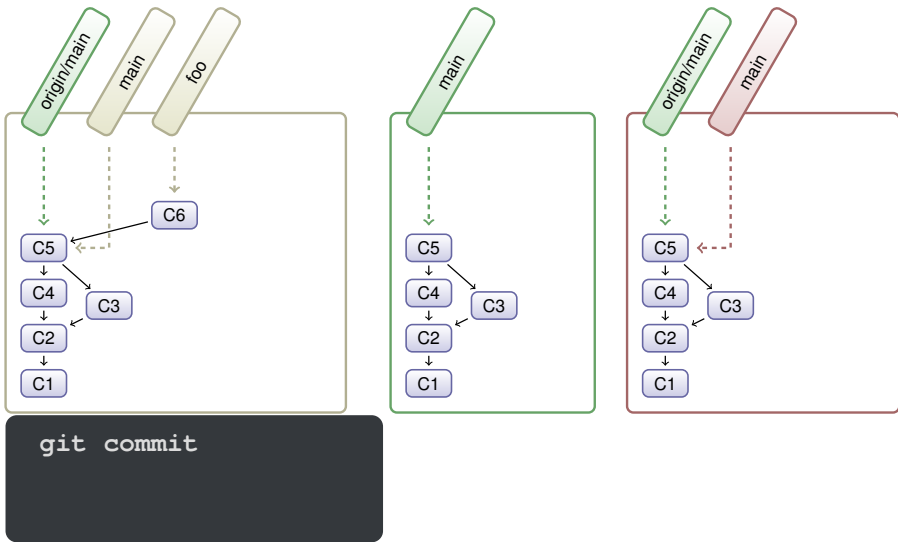
Dépôts distants et gestion de la concurrence.



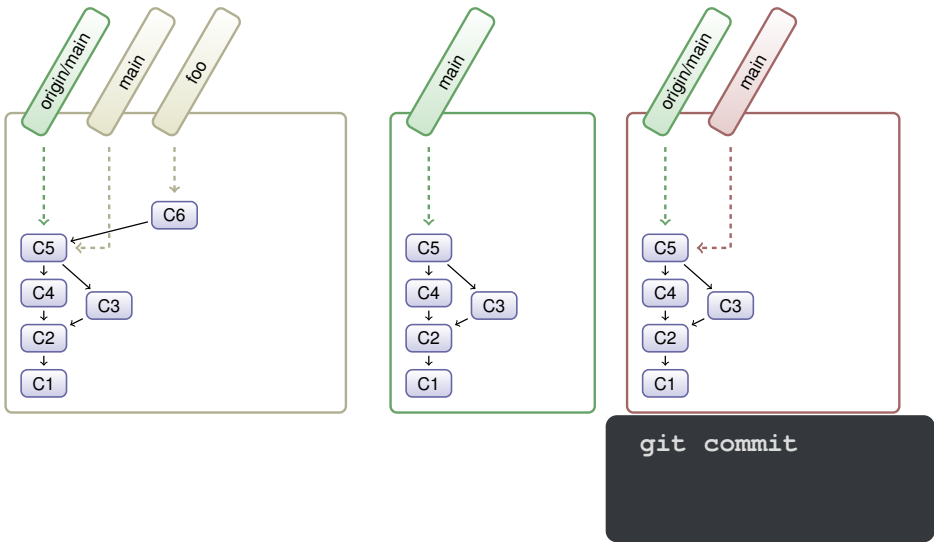
Dépôts distants et gestion de la concurrence.



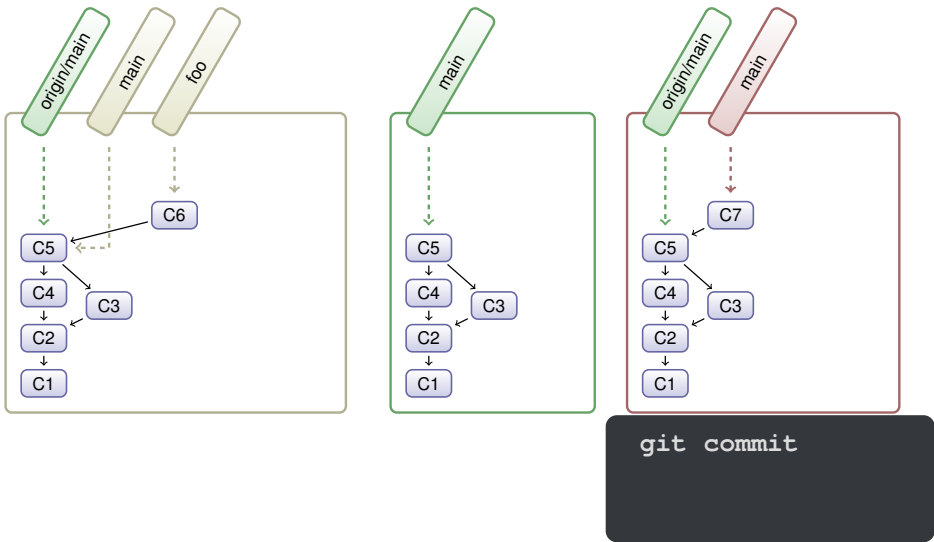
Dépôts distants et gestion de la concurrence.



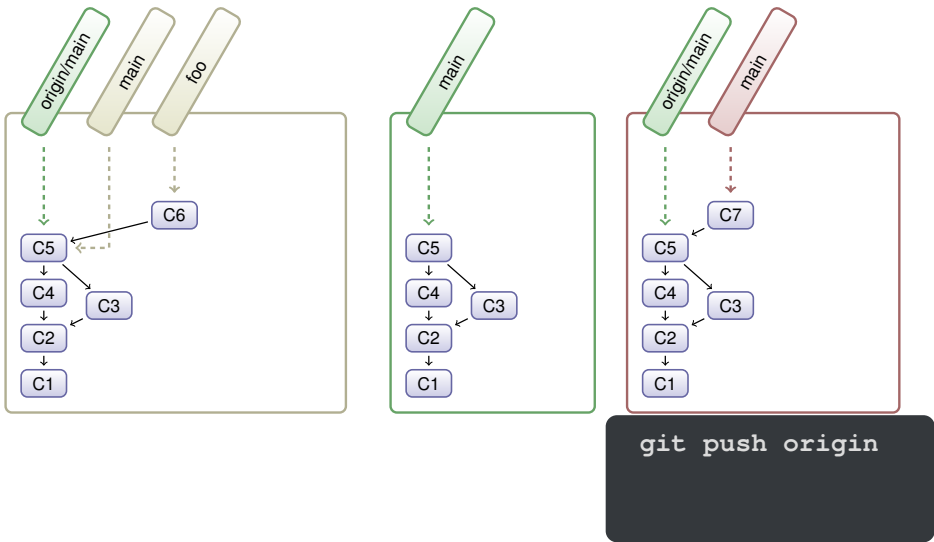
Dépôts distants et gestion de la concurrence.



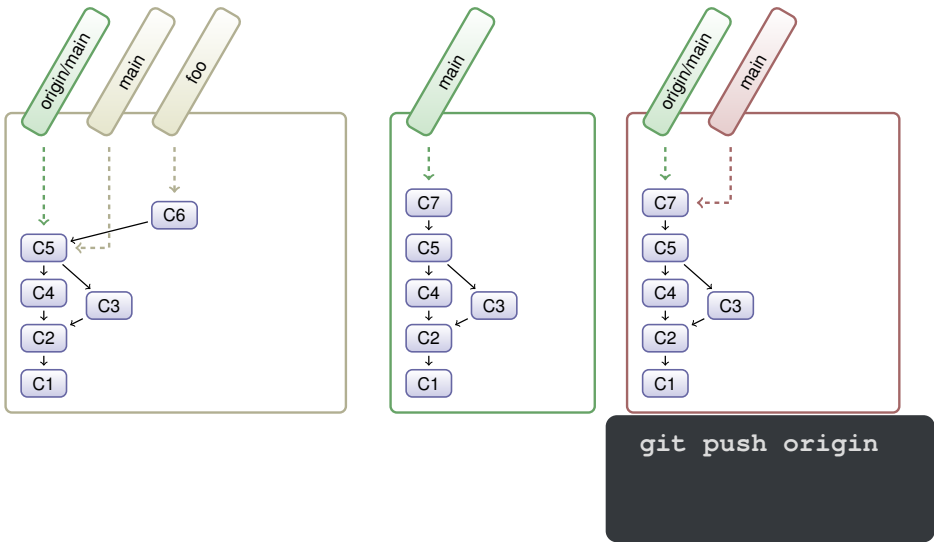
Dépôts distants et gestion de la concurrence.



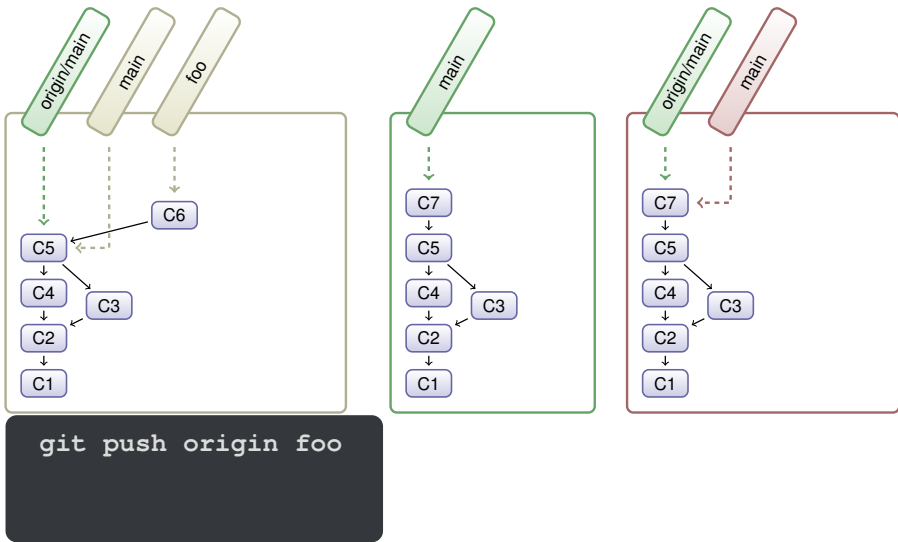
Dépôts distants et gestion de la concurrence.



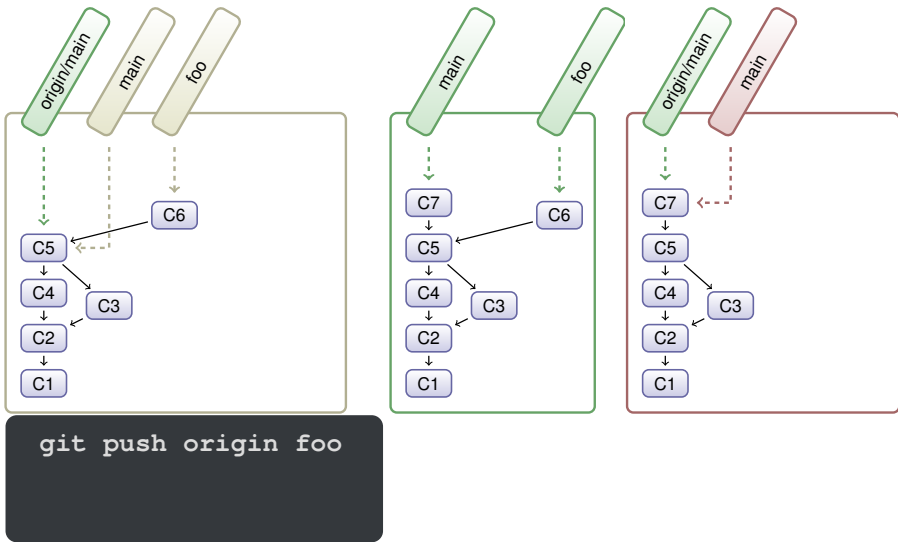
Dépôts distants et gestion de la concurrence.



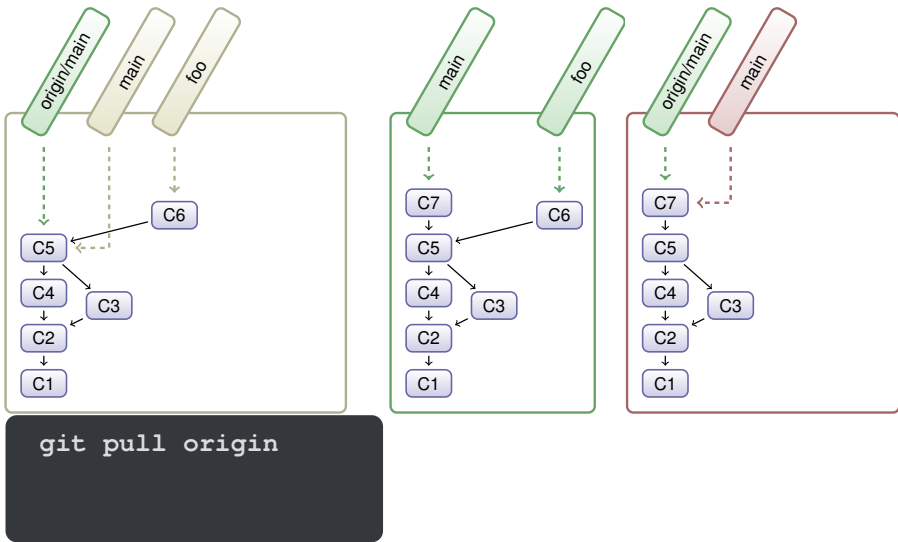
Dépôts distants et gestion de la concurrence.



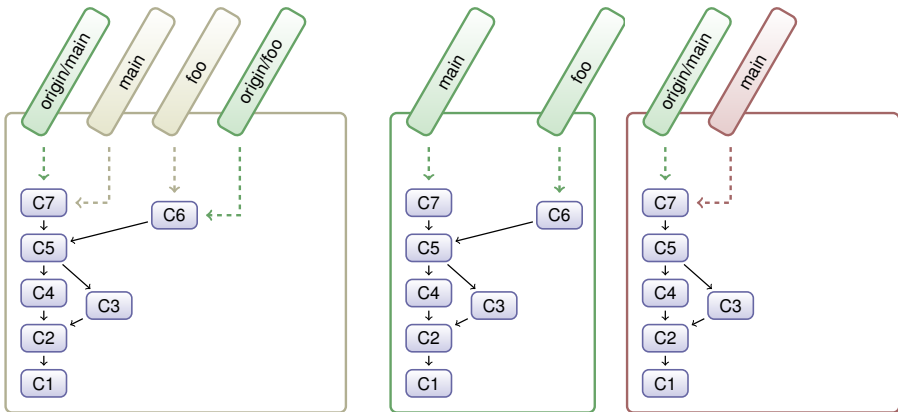
Dépôts distants et gestion de la concurrence.



Dépôts distants et gestion de la concurrence.

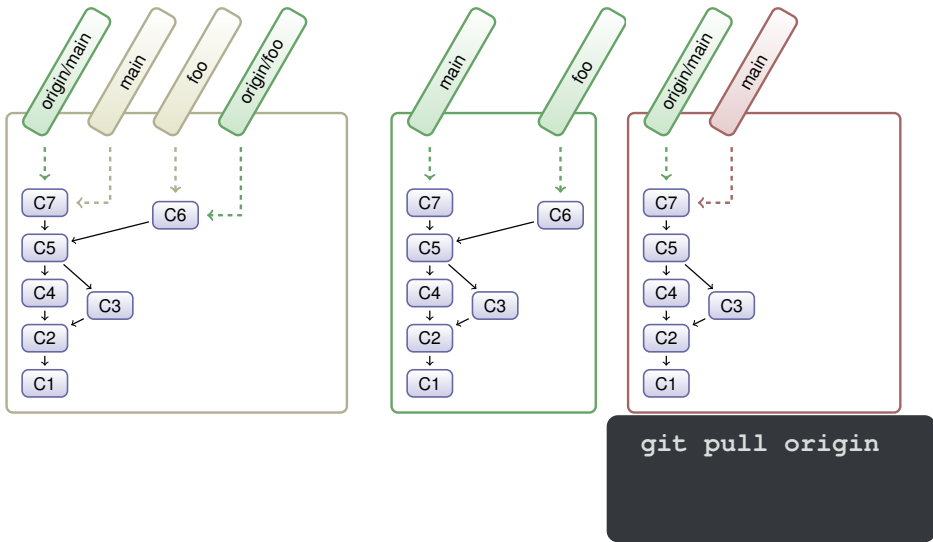


Dépôts distants et gestion de la concurrence.

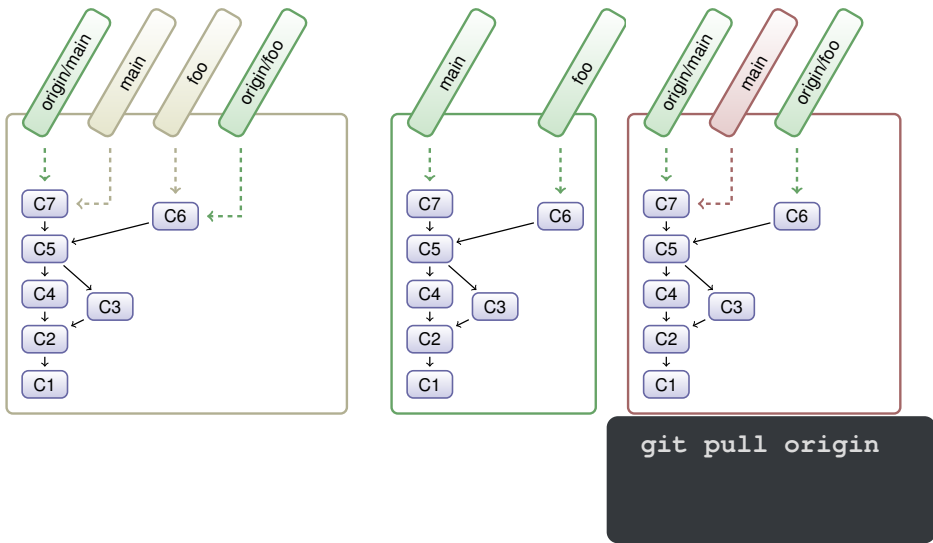


```
git pull origin
```

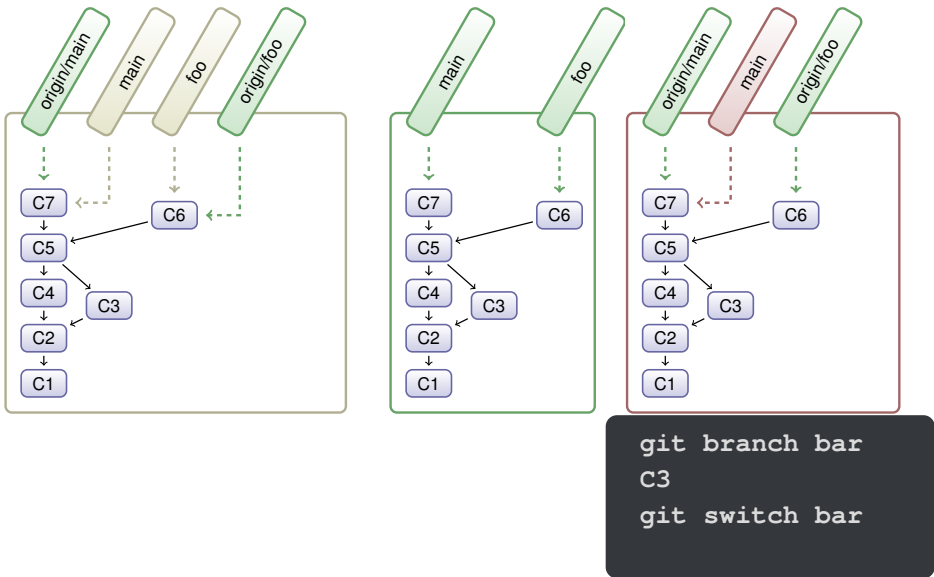
Dépôts distants et gestion de la concurrence.



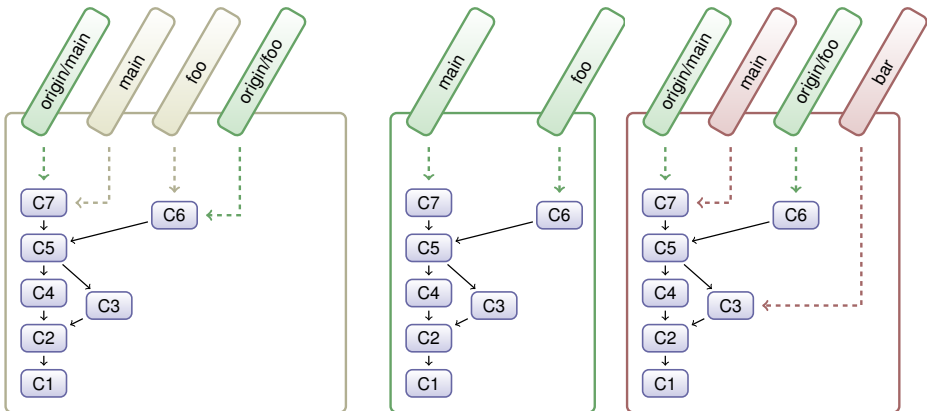
Dépôts distants et gestion de la concurrence.



Dépôts distants et gestion de la concurrence.

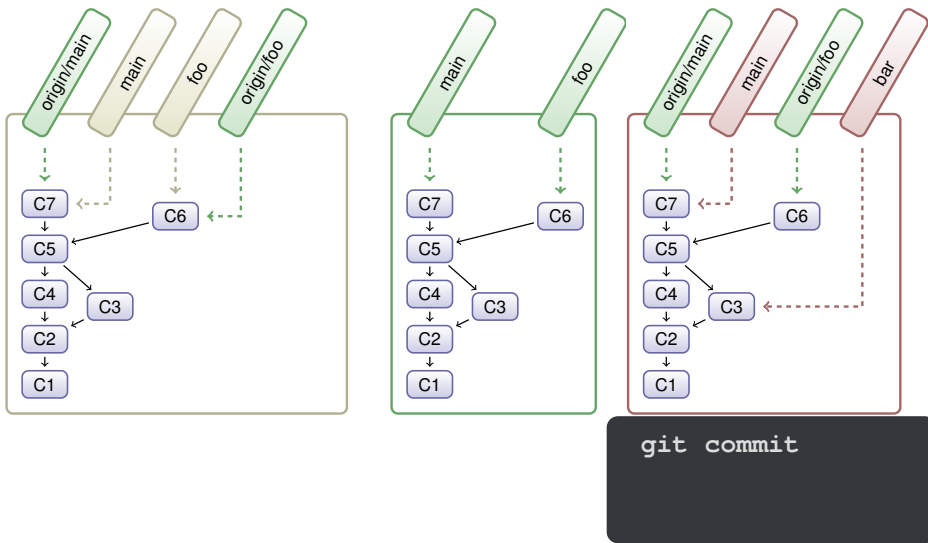


Dépôts distants et gestion de la concurrence.

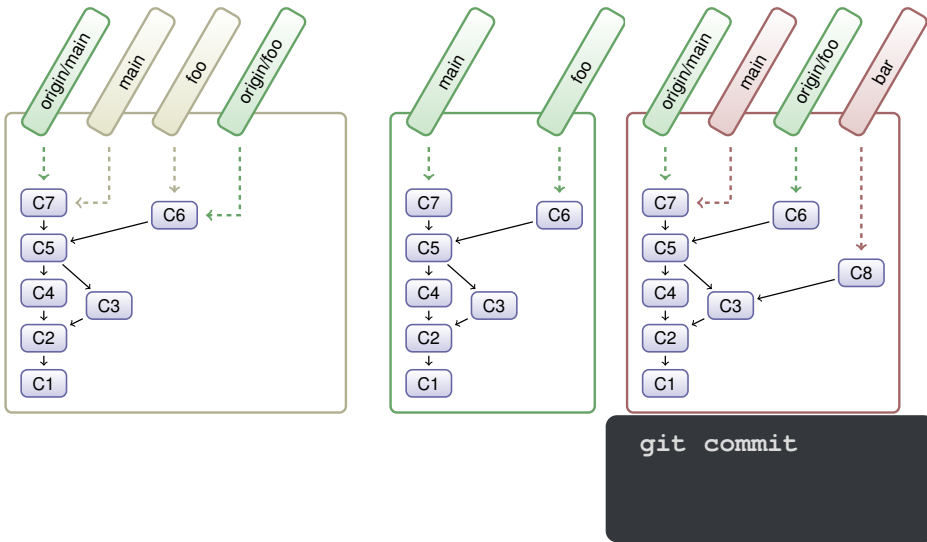


```
git branch bar  
C3  
git switch bar
```

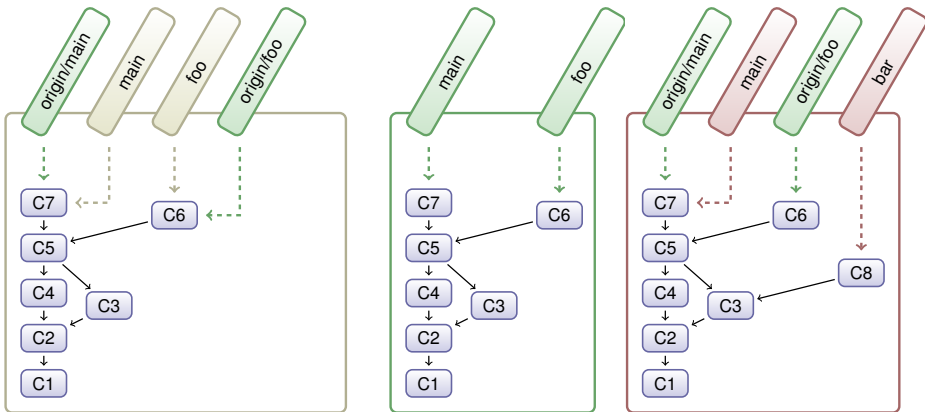
Dépôts distants et gestion de la concurrence.



Dépôts distants et gestion de la concurrence.

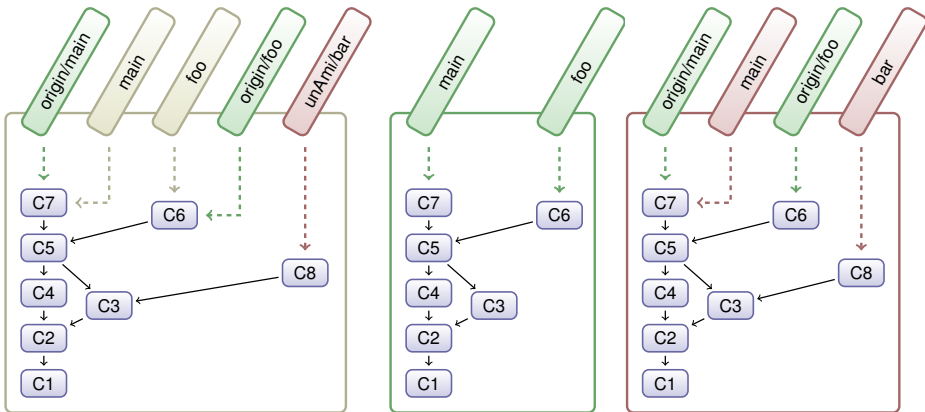


Dépôts distants et gestion de la concurrence.



```
git remote add unAmi
git://...
git fetch unAmi/bar
```

Dépôts distants et gestion de la concurrence.



```
git remote add unAmi
git://...
git fetch unAmi/bar
```

Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Principe des DVCS : Gestionnaire de versions décentralisé.

Les dépôts distants.

Modèles de travail coopératif

Les outils graphiques

Conclusion

Git permet aux développeurs de gérer leurs sources de 4 manières :

- ▶ un dépôt centralisé à la CVS, SVN tout en conservant les avantages de la conservation du dépôt local ;
- ▶ un dépôt pour chaque développeur, chacun se synchronise chez les autres, méthode traditionnelle de Arch ;
- ▶ un dépôt pour chaque développeur et manager qui se synchronise, fait les merges nécessaires et envoie le tout sur un dépôt public.
- ▶ une gestion par mails de dépôts, méthode de développement du noyau Linux et de son équipe de maintenance.

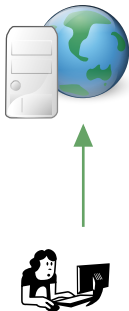
Modèle avec dépôt centralisé

"Centralized Workflow."



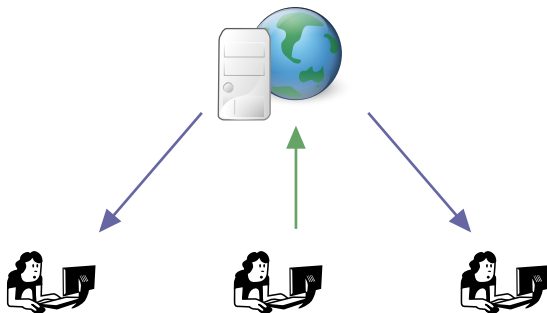
Modèle avec dépôt centralisé

"Centralized Workflow."



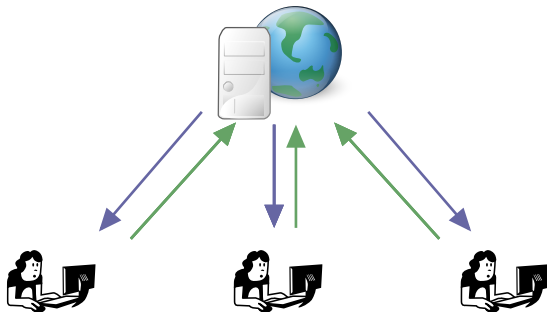
Modèle avec dépôt centralisé

"Centralized Workflow."



Modèle avec dépôt centralisé

"Centralized Workflow."



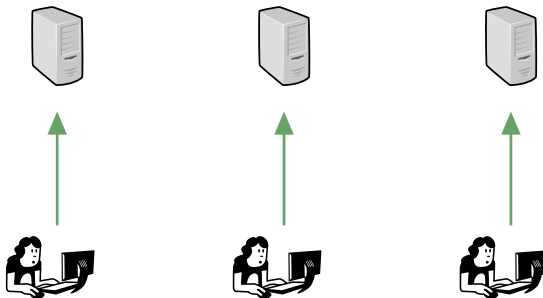
Modèle décentralisé avec dépôts publics.

"Cooperative and Decentralized Workflow."



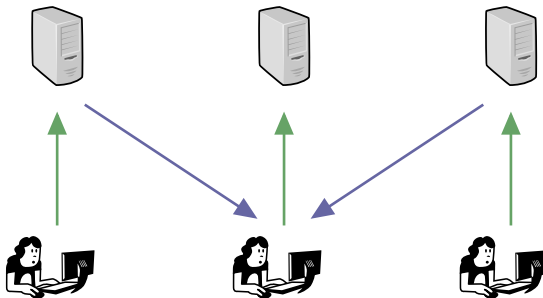
Modèle décentralisé avec dépôts publics.

"Cooperative and Decentralized Workflow."



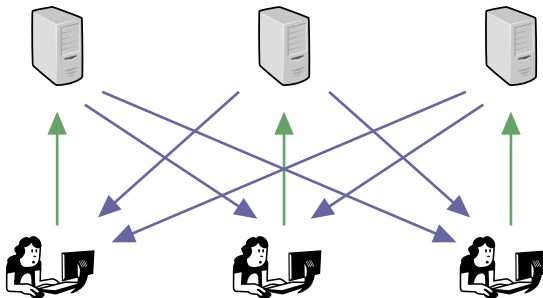
Modèle décentralisé avec dépôts publics.

"Cooperative and Decentralized Workflow."



Modèle décentralisé avec dépôts publics.

"Cooperative and Decentralized Workflow."



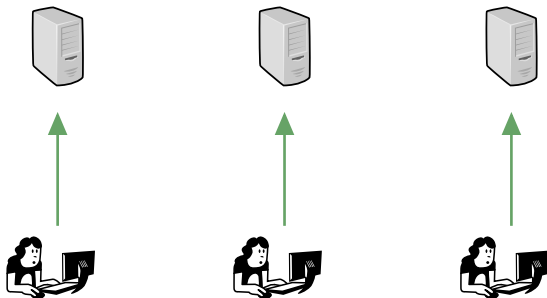
Modèle avec manager de dépôt

"Integration-Manager Workflow."



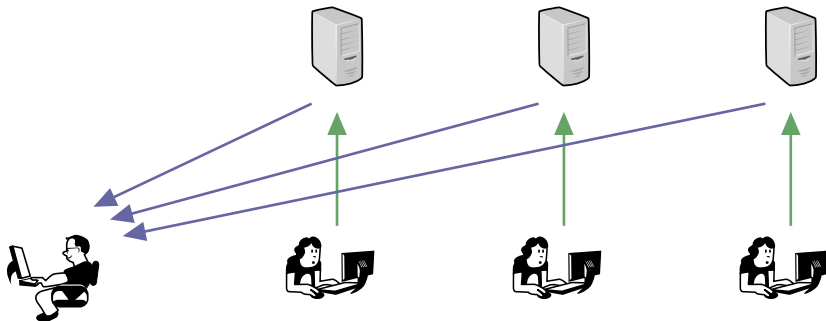
Modèle avec manager de dépôt

"Integration-Manager Workflow."



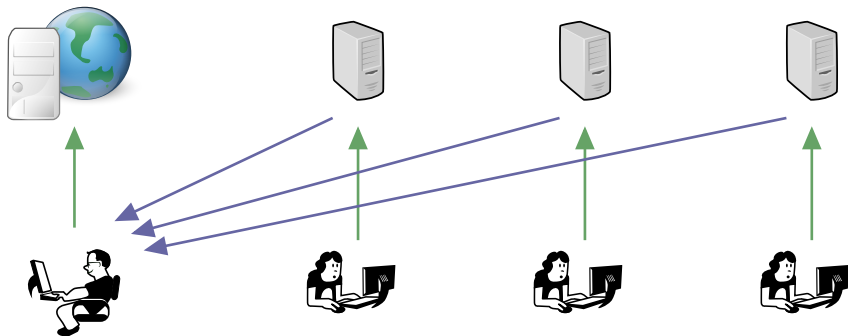
Modèle avec manager de dépôt

"Integration-Manager Workflow."



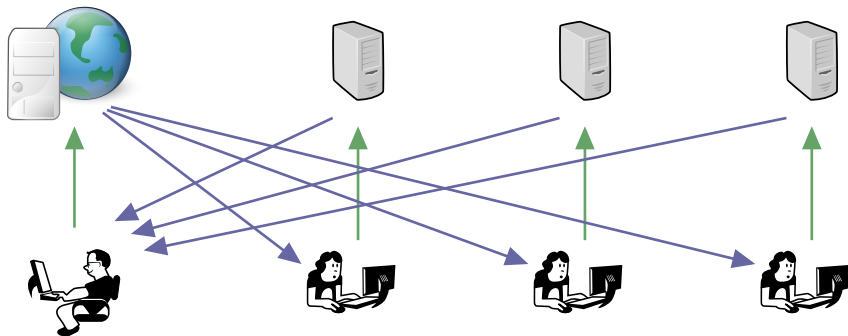
Modèle avec manager de dépôt

"Integration-Manager Workflow."



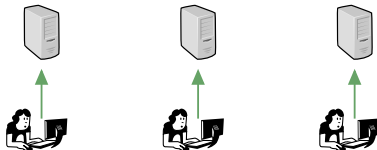
Modèle avec manager de dépôt

"Integration-Manager Workflow."



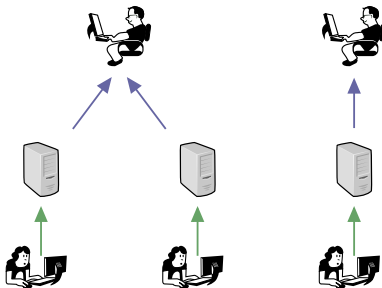
Modèle avec dictateur et lieutenants.

"Dictator and Lieutenants Workflow."



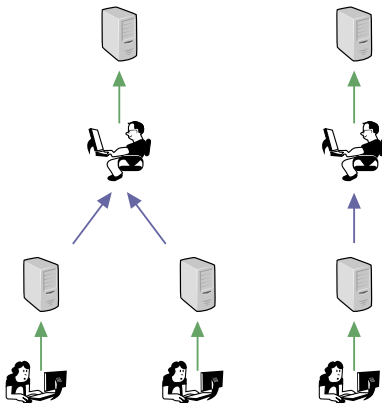
Modèle avec dictateur et lieutenants.

"Dictator and Lieutenants Workflow."



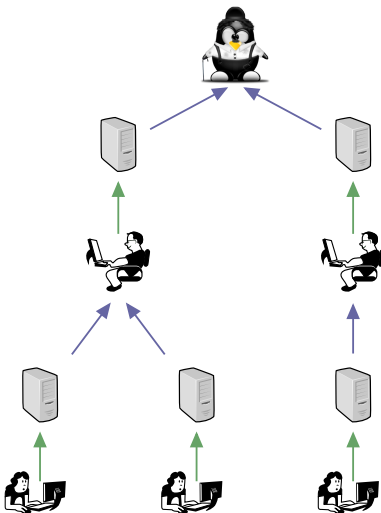
Modèle avec dictateur et lieutenants.

"Dictator and Lieutenants Workflow."



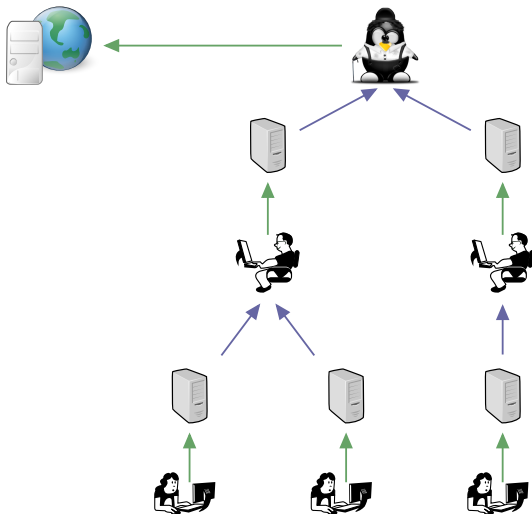
Modèle avec dictateur et lieutenants.

"Dictator and Lieutenants Workflow."



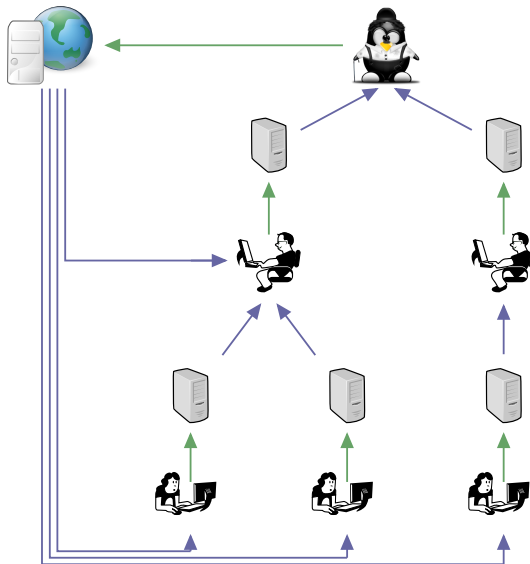
Modèle avec dictateur et lieutenants.

"Dictator and Lieutenants Workflow."



Modèle avec dictateur et lieutenants.

"Dictator and Lieutenants Workflow."



Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

git-gui et gitk

Exemple de gitg

Conclusion

Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

git-gui et gitk

Exemple de gitg

Conclusion

Il existe de nombreuses interfaces graphiques permettant de gérer vos projets quelle que soit votre plate-forme de développement :

- ▶ **Avec git :**
 - ▶ **gitk** : l'interface de visualisation détaillée et graphique d'un historique git
 - ▶ **git-gui** : outil permettant de construire les commits
- ▶ **Linux** : gitg, Giggles, ...
- ▶ **Windows** : TortoiseGit, GitExtensions, ...
- ▶ **Apple** : GitX, Gitti, ...
- ▶ **Eclipse** : EGit, ...

Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

git-gui et gitk

Exemple de gitg

Conclusion

gitg : visualisation de l'historique

The screenshot shows the gitg application window titled "gitg - git (master)". The interface includes a menu bar (Fichier, Éditer, Affichage, Dépôt, Aide), a toolbar, and a main window divided into several panes.

Historique (History) Pane: Displays a list of commits with columns for Subject, Author, and Date. The commits are color-coded by branch. The current commit is highlighted in blue.

Subject	Author	Date
Merge branch 'kf/askpass-config'	Junio C Hamano	mer. 08 sept. 2010 18:17:01 CEST
Merge branch 'bcm/maint-fetch-url-only'	Junio C Hamano	mer. 08 sept. 2010 18:17:00 CEST
Merge branch 'k/maint-pass-c-config-in-env'	Junio C Hamano	mer. 08 sept. 2010 18:17:00 CEST
Merge branch 'en/d-f-conflict-fix'	Junio C Hamano	mer. 08 sept. 2010 17:54:01 CEST
fast-export: ensure that a renamed file is printed after all references	Johannes Sixt	mar. 07 sept. 2010 21:33:02 CEST
Documentation: move RelNotes into a directory of their own	Nicolas Pitre	mar. 07 sept. 2010 02:29:57 CEST
Merge branch 'maint'	Junio C Hamano	mar. 07 sept. 2010 02:40:18 CEST
revert: Fix trivial comment style issue	Elijah Newren	lun. 06 sept. 2010 23:53:24 CEST
cache_tree_free: Fix small memory leak	Elijah Newren	lun. 06 sept. 2010 23:40:16 CEST
Merge branch 'j/v/clean-exclude'	Junio C Hamano	mar. 07 sept. 2010 01:57:05 CEST
builtin/clean.c: Use STRING_LIST_INIT_NODUP	Thiago Farina	mar. 07 sept. 2010 01:32:55 CEST
t3404 & t7508: cd inside subshell instead of around	Jens Lehmann	lun. 06 sept. 2010 20:41:06 CEST
vt.7.3reb Merge branch 'maint'	Junio C Hamano	lun. 06 sept. 2010 09:12:04 CEST
Merge branch 'w/trivial' into maint	Junio C Hamano	lun. 06 sept. 2010 09:11:59 CEST
tag.c: whitespace breakages fix	Junio C Hamano	lun. 06 sept. 2010 07:32:05 CEST
Fix whitespace issue in object.c	Jared Hance	dim. 05 sept. 2010 21:36:33 CEST
t5505: add missing &&	Jens Lehmann	dim. 05 sept. 2010 14:56:11 CEST
Merge branch 'j/s/module-ignore-diff'	Junio C Hamano	sam. 04 sept. 2010 17:17:09 CEST
Merge branch 'ab/test-2'	Junio C Hamano	sam. 04 sept. 2010 17:15:36 CEST
Merge branch 'j/s/detached-stash'	Junio C Hamano	sam. 04 sept. 2010 07:45:58 CEST

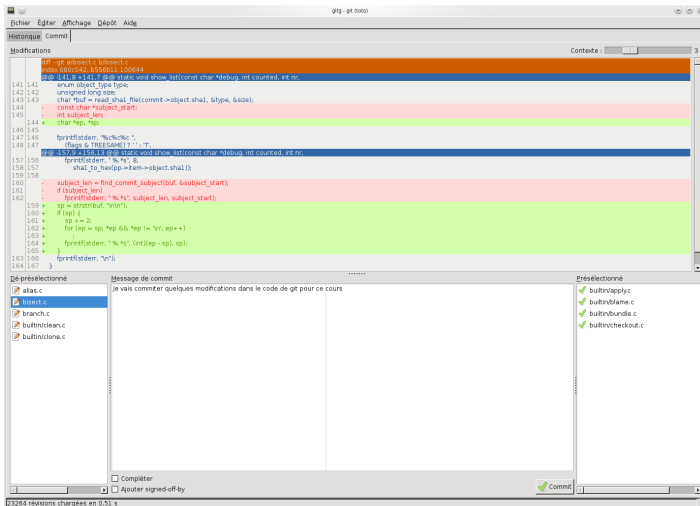
Détails (Details) Pane: Shows information for the selected commit (t3404 & t7508: cd inside subshell instead of around).

SHA: c2e0940b44ded03fa02be95c35b231fae633c1
Auteur: Jens Lehmann
Date: lun. 06 sept. 2010 20:41:06 CEST
Sujet: t3404 & t7508: cd inside subshell instead of around
Parent: 4682693e9ccc04252dfad6f5627fc056abcd8ba (Merge branch 'maint')

Diff (Diff) Pane: Shows the changes made in the selected commit. The diff is color-coded to show additions (green) and deletions (red).

```
diff --git a/t3404-rebase-interactive.sh b/t3404-rebase-interactive.sh
index af3b663..7d20a74 100755
@@ -101,10+101,10 @@ test_expect_success 'rebase -i with the exec command' {
     test_expect_success 'rebase -i with the exec command runs from tree root' {
         git checkout master &&
         mkdir subdr && cd subdr &&
         mkdir subdr && cd subdr &&
         false || exec &touch-subdr\*
         git rebase -i HEAD~ &&
     }
```

gitg : commit interactif



Git c'est quoi ?

Architecture interne de git

Gestion des versions dans le dépôt local.

Synchronisation avec les dépôts distants.

Les outils graphiques

Conclusion

Git vs Rhinocéros

Mercurial, Darcs, Bazaar, Arch, etc

Les points forts :

- + Certainement le plus rapide à appliquer des patches
- + Simple à mettre en place (mode sans serveur)
- + Petits outils puissants (esprit Unix)
- + Git est distribué sous licence GNU GPL 2
- + Développement actif
- + Linus

Les points faibles :

- Courbe d'apprentissage
- Linus

- ▶ Documentation officielle et usuelle : <http://www.kernel.org/pub/software/scm/git/docs/everyday.html>
- ▶ Documentation officielle complète : <http://www.kernel.org/pub/software/scm/git/docs/>
- ▶ Manuel de référence communautaire et en français : <http://alexgirard.com/git-book/index.html>
- ▶ Manuel de référence communautaire (ultra complet) et en anglais : <http://progit.org/book/>
- ▶ Tutoriel en video et en anglais : <http://www.gitcasts.com/>
- ▶ Linus Torvalds parle de git chez *Google* (1 heure environ) : <http://www.youtube.com/watch?v=4XpnKHJAok8>
- ▶ N'oubliez pas les *manpages*...

- ▶ <http://download.ikaaro.org/doc/git/200607-ols.pdf>
- ▶ <http://kernel.org/pub/software/scm/cogito/>
- ▶ <http://git.or.cz/>
- ▶ <http://git.or.cz/gitwiki/>
- ▶ <http://www.kernel.org/pub/software/scm/git/docs/everyday.html>
- ▶ <http://www.kernel.org/pub/software/scm/git/docs/>